



UNIVERSITI SAINS MALAYSIA

School
of
Electrical and Electronic Engineering

EEM 441

**Control, Instrumentation
& FPGA Laboratory**

Laboratory Manual

Experiments 1–10

Contents

Experiment 1: DC Motor / PID Control — Speed	2
Experiment 2: DC Motor / PID Control — Position.....	8
Experiment 3: Introduction to Altera Quartus II Software, and Interfacing FPGA to DIP Switch and Seven Segment	15
Experiment 4: LCD Interfacing to Altera DE2 Board, and Stepper Motor Control with FPGA	25
Experiment 5: Interfacing FPGA to Analog-to-Digital Converter.....	42
Experiment 6: Interfacing FPGA to Digital-to-Analog Converter.....	47
Experiment 7: DC Motor Speed Measurement System	51
Experiment 8: Temperature Control System	58
Experiment 9: Project Creation in SCADA (IoT)	63
Experiment 10: Dashboard Visualization and Analysis (IoT)	70





UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 1

DC MOTOR / PID CONTROL — SPEED

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 1

DC Motor / PID Control — Speed

Objectives

1. To study the open-loop speed control of a DC motor.
2. To study closed-loop speed control using P, PD, PI, and PID controllers.

Equipment / Software Required

- Controller kit
- Cathode ray oscilloscope
- BNC connectors with leads
- Multimeter

Procedure

(a) Open-loop control of DC motor

1. Switch on the supply and measure all constant voltages; calibrate variable voltages in terms of voltage and angle (degrees).
2. Disconnect all cable wires from the hardware module (Figure 1.1).
3. Connect *Output A* of the motor plant to Channel 2 of the oscilloscope.
4. Connect *Adjust A* of the motor plant to *Input A* (speed control of the plant) and to Channel 1 of the oscilloscope (only positive voltages are considered).
5. Turn ON the power switch.
6. Slowly turn potentiometer-A clockwise (0 to +50) and anti-clockwise (0 to -50). Observe and describe the motor operation.
7. Disconnect *Adjust A* and connect the motor *Input A* to *Adjust B* (only positive voltages are considered).
8. Vary supply B from 30° to 270° in steps of 30° and study the variation in reference speed (voltage) and output speed (voltage). Plot the error curve.
9. Comment on your observations and results.

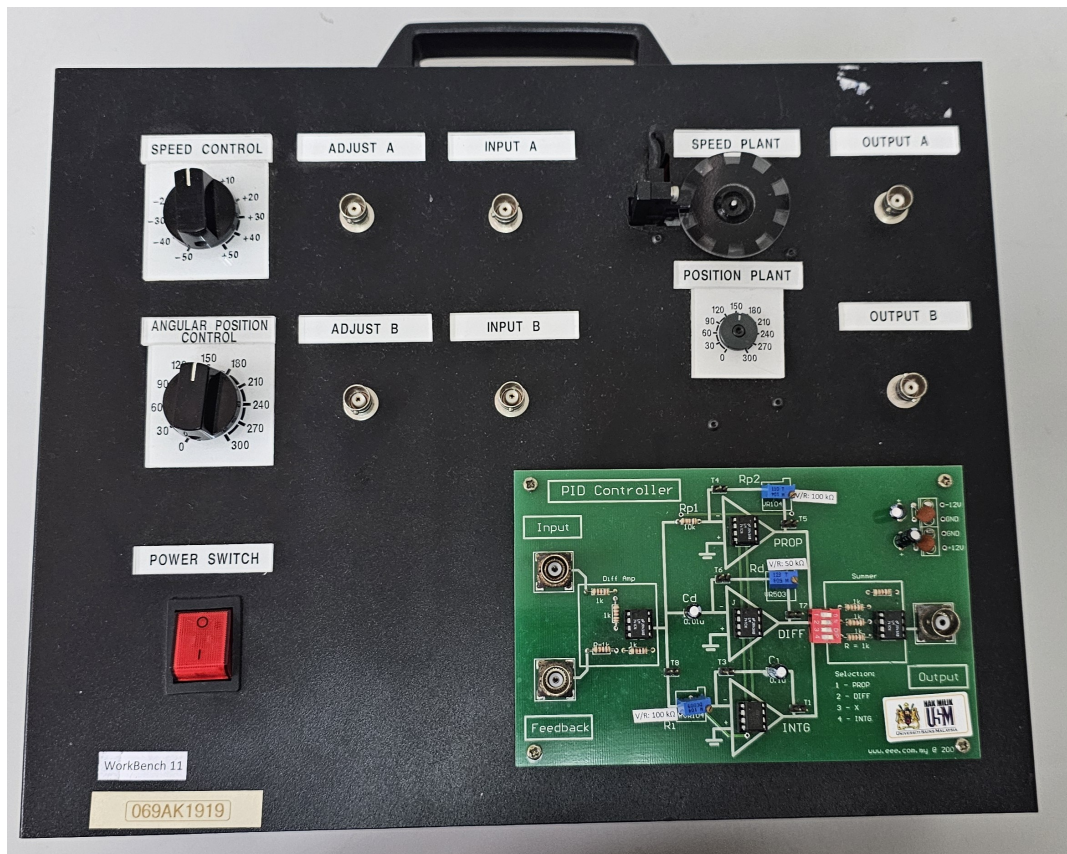


Figure 1.1: DC Motor / PID Control trainer.

Table 1.1: Open-loop speed response.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
+270				
+240				
+210				
+180				
+150				
+120				
+90				
+60				
+30				

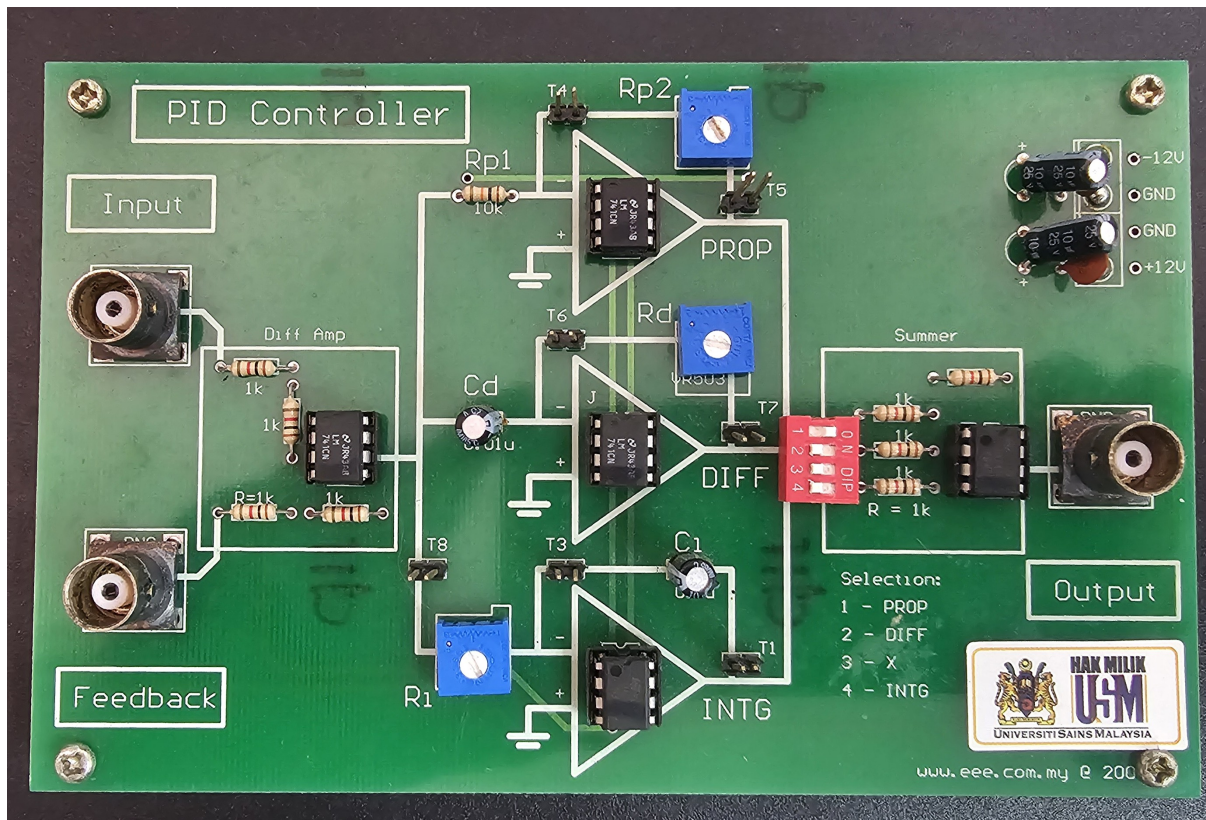


Figure 1.2: PID Control module.

(b) Closed-loop control of speed of DC motor

(i) Using P-controller with feedback through optical encoder

1. Select the P-controller by setting PROP on the controller (DIP switch 1) to ON (refer to Figure 1.2).
2. Connect *Adjust B* of the plant to the input of the PID controller.
3. Connect *Output A* of the plant to the controller feedback.
4. Connect the controller output to *Input A* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. +180).
6. Adjust trimmer R_{p2} (varies K_p) to obtain an optimum response (minimum steady-state error).
7. Turn potentiometer-B to vary the speed of the motor. Record the input and output voltages for various positions in tabular form.

(ii) Speed control with PI-controller

1. On the PID controller card, set trimmers R_p and R_d to minimum (counter-clockwise) while R_i is set to maximum (clockwise).
2. Select the PI-controller by setting PROP and INTG to ON.
3. Slowly vary trimmer R_i to obtain an optimum output response (minimum steady-state error). If gain is insufficient (or overshoots), adjust R_{p2} .

4. Turn potentiometer-B to vary the speed of the motor. Record the input and output voltages for various positions (Table 1.2). Repeat step 3 if necessary to fine-tune the response.
5. Compare the experimental results for the P-controller and the PI-controller.

(iii) Speed control with PD-controller

1. Switch on the supply and measure all constant voltages; calibrate variable voltages in terms of voltage and angle.
2. On the PID controller card, set trimmers R_p and R_d to minimum (counter-clockwise) while R_i is set to maximum (clockwise).
3. Select the PD-controller by setting PROP and DIFF to ON; all other switches OFF.
4. Vary supply B from 30° to 270° in steps of 30° and study the variation in reference speed (V_{in}) and output speed (V_o). Plot the error curve.
5. Study the effect of varying R_d on the output.

(iv) Speed control with PID-controller

1. Adjust potentiometer-B to 2 V (around $+180^\circ$).
2. Set trimmer R_{p2} to minimum to reduce the damping ratio.
3. Set trimmer R_i to minimum. The output should start to vibrate.
4. Slightly increase R_i until the system starts oscillating between two points (pure oscillation).
5. Slightly increase R_{p2} to bring the system to a damped condition; it should oscillate for a few cycles.
6. Apply a small force to hold the motor (the oscilloscope will show the motor output drop to 0 V); observe and describe the response when the force is released.
7. Select the PID-controller by setting PROP, INTG, and DIFF to ON.
8. Slowly vary R_d to obtain the optimum response.
9. Compare its operation with the P, PI, and PD controllers under disturbance.
10. As before, take observations by varying the applied voltage.

Table 1.2: Closed-loop speed response.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
+270				
+240				
+210				
+180				
+150				
+120				
+90				
+60				
+30				

Report

1. Compare the performance of P, PI, PD, and PID controllers.
2. What will happen if all types of controllers are connected in series?
3. What are the different types of encoders? Explain the working principle of an optical encoder with the help of suitable diagrams.



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 2

DC MOTOR / PID CONTROL — POSITION

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 2

DC Motor / PID Control – Position

Objectives

1. To study open-loop position control of a DC motor.
2. To study feedback (closed-loop) control using P, PI, PD, and PID controllers.
3. To implement position control of a DC motor using a potentiometer and a resistance-to-voltage converter.

Equipment / Software Required

- Controller kit
- Cathode ray oscilloscope
- BNC connectors with leads
- Multimeter

Procedure

(a) Position control without feedback

1. Connect *Output B* of the motor plant to Channel 2 of the oscilloscope.
2. Connect *Adjust A* of the motor plant to *Input B* (position control) of the plant and to Channel 1 of the oscilloscope.
3. Turn on the power switch.
4. Slowly turn potentiometer-A clockwise (0 to +50) and anti-clockwise (0 to –50). Observe and describe the motor operation. Can you control the angular position of the motor?
5. Turn off the motor by disconnecting *Input B*.

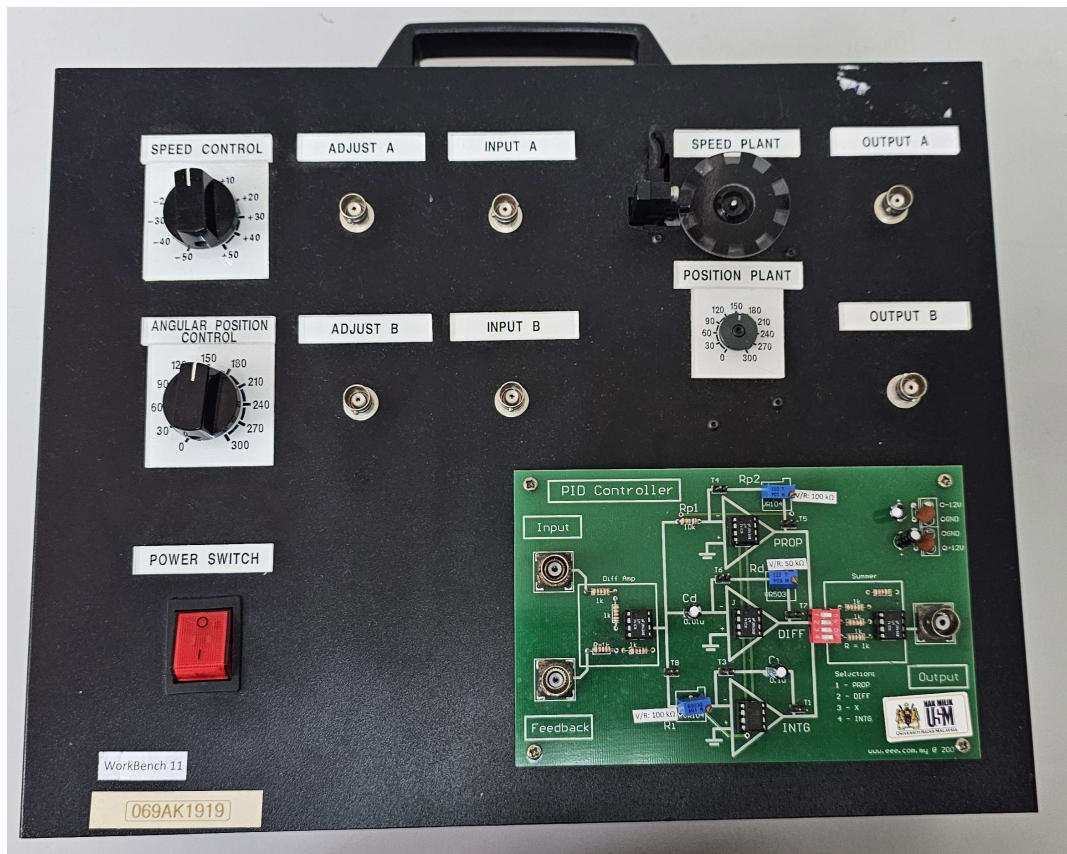


Figure 2.1: DC Motor / PID Control trainer.

(b) Closed-loop position control

(i) Position control with proportional feedback control

The motion of a motor can be converted into rotational position with the help of a suitable gearbox. The electrical signal, proportional to (angular) position, is obtained with the help of a circular potentiometer of good accuracy.

This signal is fed back to the summer to extract an error signal, which is fed to the controller to obtain optimum performance. Different controllers may be tried for this case, to study them: use the PID Controller daughterboard shown in Figure 2.2, selecting the desired stage(s) via its DIP switch.

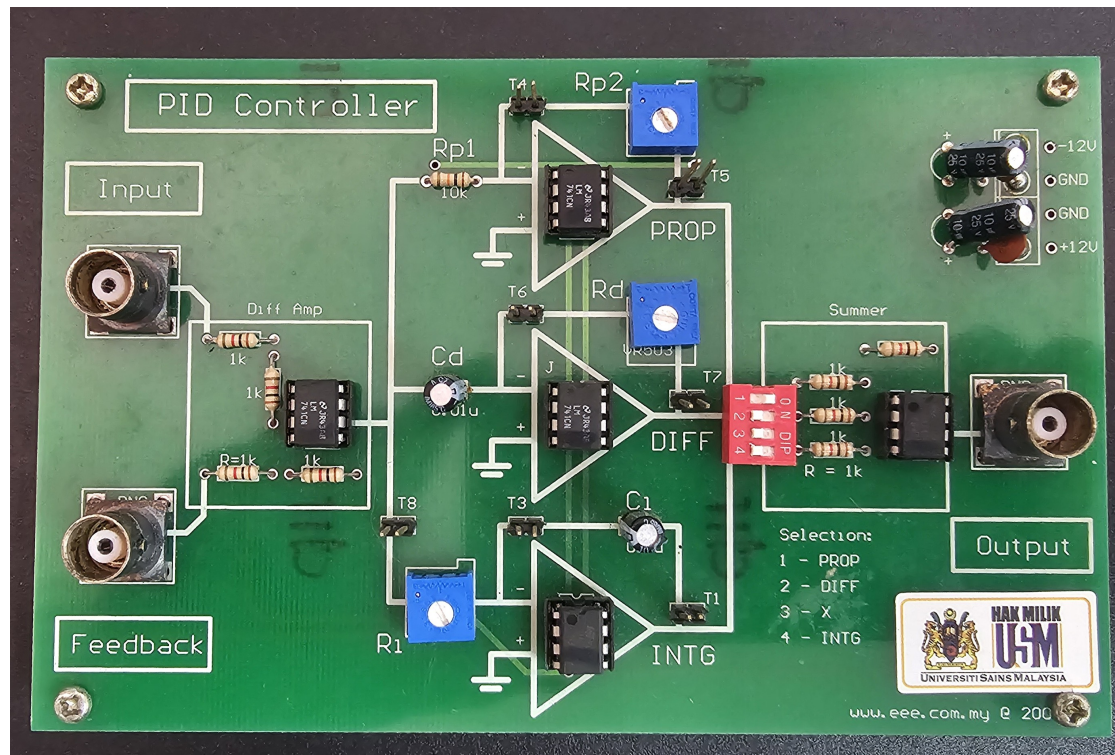


Figure 2.2: PID Control module.

1. Select the P-controller by setting PROP on the controller (DIP switch 1) to ON; all other switches OFF.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. +180).
6. Adjust trimmer R_{p2} (varying K_p) to obtain an optimum response (minimum steady-state error).
7. Turn potentiometer-B to vary the position of the motor. Can the position be controlled in this case?
8. Record the input and output voltages for various positions in Table 2.1.

Table 2.1: Position control response.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
+270				
+240				
+210				
+180				
+150				
+120				
+90				
+60				
+30				

(ii) Position control with feedback and PI-controller

1. Select the PI-controller by setting PROP and INTG to ON.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. +180).
6. Slowly vary the respective trimmers R_{p2} and R_i to obtain an optimum output response.
7. Turn potentiometer-B to vary the position of the motor. Record the input and output voltages for various positions.

(iii) Position control with PD-controller

1. Select the PD-controller by setting PROP and DIFF to ON; all other switches OFF.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. +180).
6. Slowly vary the respective trimmers R_{p2} and R_d to obtain an optimum output response.
7. Turn potentiometer-B to vary the position of the motor. Record the input and output voltages for various positions.

(iv) Closed-loop control of position of motor using PID controller

1. Select the PID-controller by setting PROP, INTG, and DIFF to ON.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. +180).
6. Adjust the value of trimmers R_{p2} , R_i , and R_d to obtain an optimum response (minimum steady-state error, quick operation, without overshoot or oscillations).
7. Turn potentiometer-B to vary the position of the motor.
8. Record the input and output voltages for various positions in Table 2.2.

Table 2.2: Closed-loop PID position response.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
+270				
+240				
+210				
+180				
+150				
+120				
+90				
+60				
+30				

Report

1. Briefly explain the working of circuits used for the conversion of resistance to voltage.
2. Describe in detail the applications of position control.



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 3

FPGA — INTRODUCTION TO ALTERA QUARTUS II
SOFTWARE, AND INTERFACING FPGA TO DIP SWITCH AND
SEVEN SEGMENT

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 3

Introduction to Altera Quartus II Software, and Interfacing FPGA to DIP Switch and Seven Segment

Objectives

1. To use schematic entry as a design method in Altera Quartus II.
2. To use Verilog as a design entry method in Altera Quartus II.
3. To combine schematic and Verilog as a single design entry in Altera Quartus II.
4. To simulate the circuit using Altera Quartus II.
5. To download the design onto an Altera DE2 board.
6. To interface a DIP switch and a seven-segment display to the FPGA.

Equipment / Software Required

1. Altera Quartus II software
 - version 9/10/12 for Cyclone II
 - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Function generator

Introduction

Seven-segment displays are widely used to show numbers. The DE2 board provides eight seven-segment displays, 18 DIP switches, four push-button switches, and 16 LEDs. These components are hardwired on the board and are each assigned to specific FPGA pins — refer to the DE2 user manual for the full pin map.

Procedure

Activity 1 — Schematic entry

Objective: use schematic entry to design a 4-to-1 multiplexer in Quartus II.

1. Open Quartus II and start the New Project Wizard.
 - Working directory: fpga; project name: activity1; top-level entity: mux_sch.
 - Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N).

- Accept the default EDA tool settings (synthesis, simulation, timing analysis) and finish.
2. Create a new Block Diagram/Schematic file.
 3. Place three 3-input AND gates and one 4-input OR gate from the primitives library, and wire them as shown in Figure 3.1 using rubberbanding to connect components.
 4. Add and name the input pins (i0–i3, s0, s1) and the output pin (out).
 5. Save as mux_sch.bdf and start compilation.
 6. Create a Vector Waveform File to simulate the design: insert input nodes i0–i3, s0, s1, and output node out.
 7. Assign logic values to the inputs, save as mux_sch.vwf, then run Compilation and Simulation.
 8. Record the outputs, then repeat for each input combination in Table 3.1, saving the waveform file each time.
 9. Print and submit the schematic diagram and simulation result.

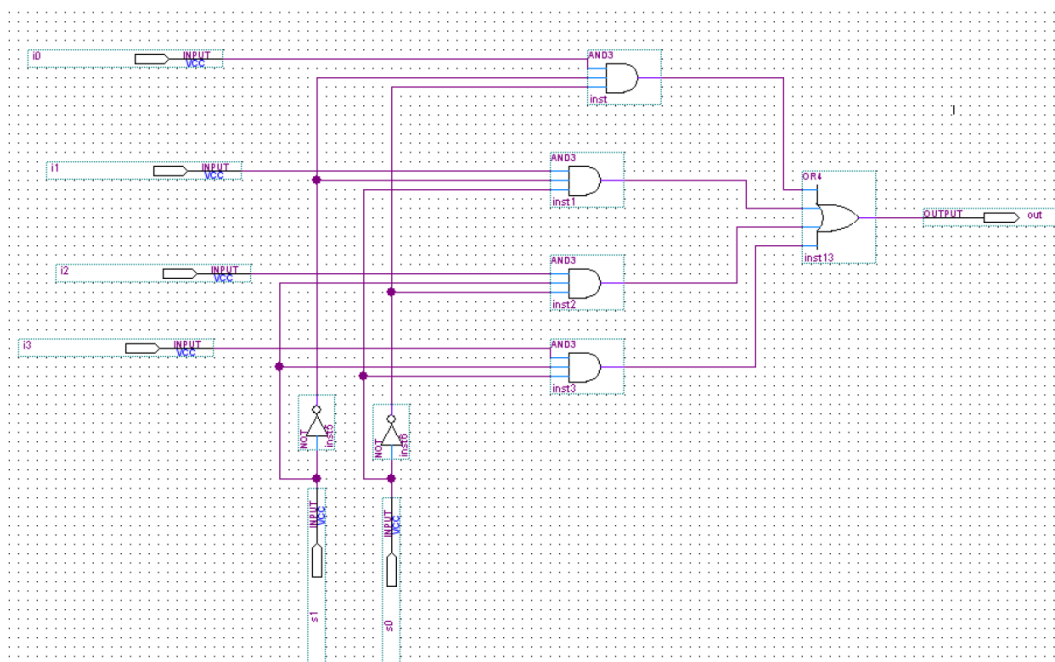


Figure 3.1: Mux Schematic.

Table 3.1: Input combinations to simulate.

S0	S1	I0	I1	I2	I3
0	0	1	1	0	0
0	1	0	1	1	0
1	0	1	1	0	1
1	1	1	1	0	0

Activity 2 – Verilog entry

Objective: use Verilog code as a design entry method in Quartus II.

1. Start a new project (Activity2, top level mux_ver) with the same device and EDA tool settings as Activity 1.
2. Create a new Verilog file and enter the code in Listing 3.1.
3. Save as mux_ver, then compile the design.
4. Create a Vector Waveform File; insert input nodes i0–i3, s0, s1, and output node out.
5. Assign logic values matching Table 3.1, save as mux_ver.vwf, then start simulation.
6. Inspect the result in the Simulation Report – Simulation Waveform.
7. Change the input values (saving before each run) and re-simulate.
8. View the synthesized schematic via **Tools** → **Netlist Viewer** → **RTL Viewer**, shown in Figure 3.2.
9. Print and submit the Verilog code and simulation result.

```
mux_ver.v
1  module mux_ver(out, i0, i1, i2, i3, s1, s0);
2      output out;
3      input i0, i1, i2, i3;
4      input s1, s0;
5      reg out;
6
7      always @(s1 or s0 or i0 or i1 or i2 or i3)
8      begin
9          case ({s1, s0})
10             2'b00: out = i0;
11             2'b01: out = i1;
12             2'b10: out = i2;
13             2'b11: out = i3;
14             default: out = 1'bx;
15          endcase
16      end
17  endmodule
```

Listing 3.1: mux_ver.v — Verilog 4-to-1 multiplexer.

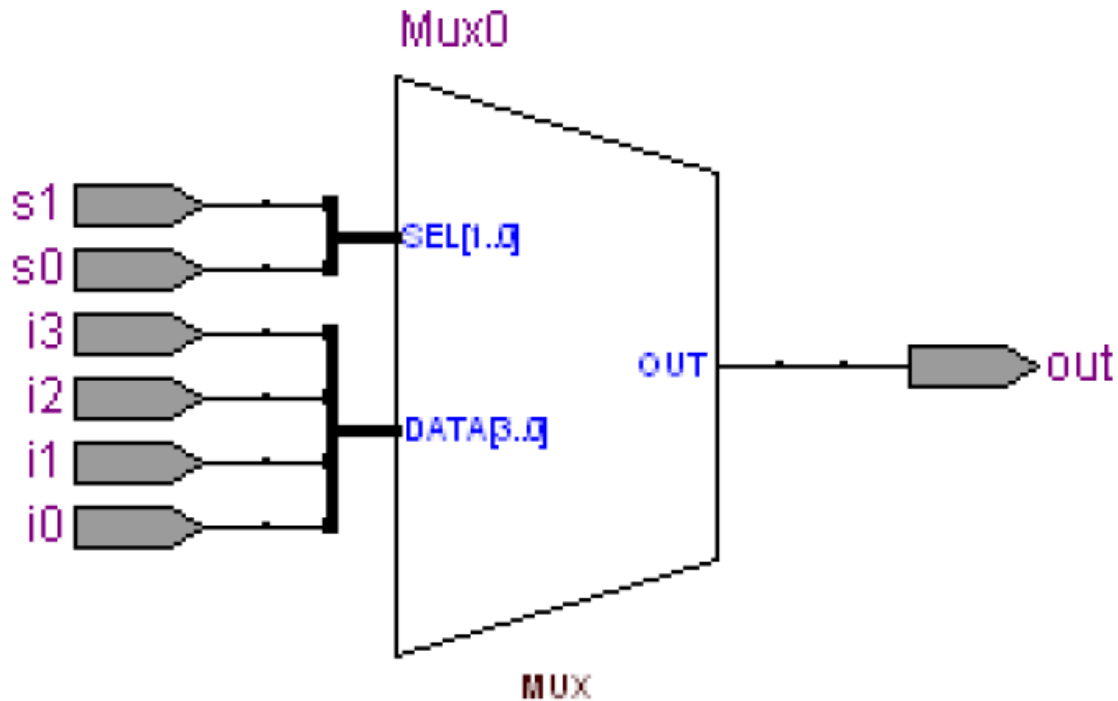


Figure 3.2: RTL Viewer output for mux_ver (Mux0).

Activity 3 – Combining schematic and Verilog entry

Objective: combine a Verilog module with schematic entry in a single design.

1. Start a new project (activity3, top level mux_schver) with the same device and EDA tool settings.
2. Create a new Verilog HDL file with the code in Listing 3.2; save as mux_vr1.
3. Generate a reusable symbol from the file via **File** → **Create/Update** → **Create Symbol Files for Current File**. The resulting block is shown in Figure 3.3.
4. Create a new Block Diagram/Schematic file, place the mux_vr1 symbol from the project library, and connect its inputs and output to input/output pads as shown in Figure 3.4.
5. Save as mux_schver.bdf and start compilation.
6. Create a Vector Waveform File, insert the same nodes as before, assign logic values, save as mux_schver.vwf, and run Compilation and Simulation.
7. Record the outputs for the combinations in Table 3.1, saving the waveform file each time.
8. Print and submit the schematic diagram, Verilog code, and simulation result.

```

mux_vrl.v
1  module mux_vrl(out, i0, i1, i2, i3, s1, s0);
2      output out;
3      input i0, i1, i2, i3;
4      input s1, s0;
5      reg out;
6
7      always @(s1 or s0 or i0 or i1 or i2 or i3)
8          begin
9              case ({s1, s0})
10                 2'b00: out = i0;
11                 2'b01: out = i1;
12                 2'b10: out = i2;
13                 2'b11: out = i3;
14                 default: out = 1'bx;
15             endcase
16         end
17     endmodule

```

Listing 3.2: mux_vrl.v — Verilog module to be reused as a schematic symbol.

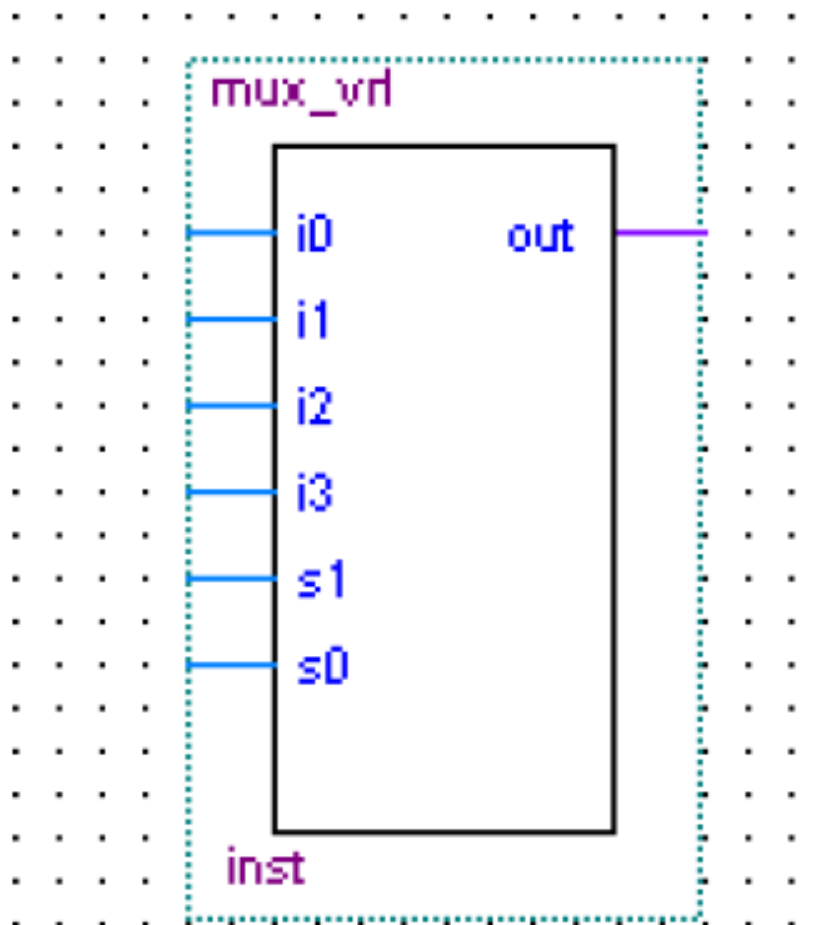


Figure 3.3: Auto-generated mux_vrl symbol block.

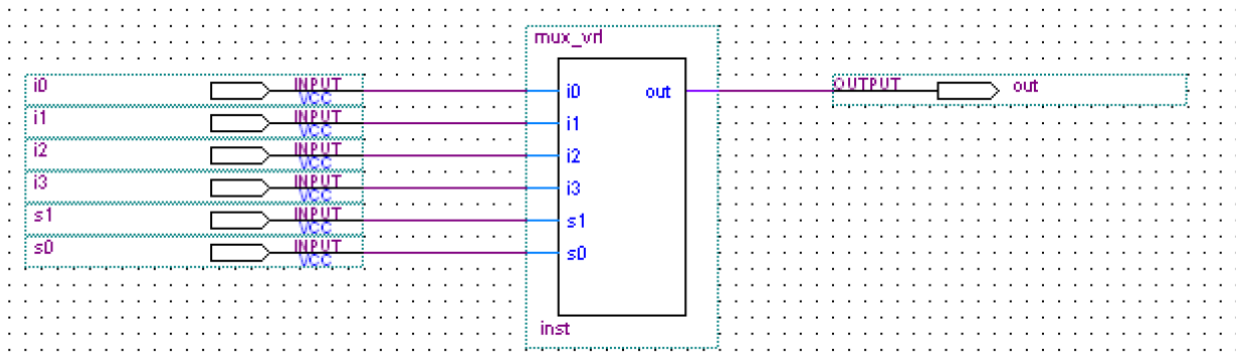


Figure 3.4: Combined schematic mux_schver.

Activity 4 – Timing analysis

Objective: analyse the maximum operating speed of a design.

1. Start a new project (Activity4, top level counter) with the same device and EDA tool settings.
2. Create a new Verilog file with the code in Listing 3.3; save as counter and compile.
3. Create a Vector Waveform File; insert cclock and cclear as inputs and Q as output.
4. Force cclear low, assign a clock signal to cclock, save as counter.vwf, then start simulation.
5. Inspect the result in the Simulation Report – Simulation Waveform.
6. Run **Processing** → **Start** → **Start Classic Timing Analyzer**.
7. Read the maximum clock frequency from the Timing Analyzer Summary (Figure 3.5).
8. Print and submit the Verilog code and simulation result.

counter.v

```

1  module counter(Q, clock, clear);
2      output [3:0] Q;
3      input clock, clear;
4      reg [3:0] Q;
5
6      always @(posedge clear or negedge clock)
7      begin
8          if (clear)
9              Q = 4'd0;
10         else
11             Q = (Q + 1) % 16;
12     end
13 endmodule

```

Listing 3.3: counter.v – 4-bit up counter with asynchronous clear.

Timing Analyzer Summary									
	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tco	N/A	None	7.410 ns	Q[3]~reg0	Q[3]	clock	--	0
2	Clock Setup: 'clock'	N/A	None	Restricted to 233.64 MHz (period = 4.280 ns)	Q[0]~reg0	Q[3]~reg0	clock	clock	0
3	Total number of failed paths								0

Figure 3.5: Timing analyzer summary for the counter design.

Activity 5 – Downloading to the DE2 board

Objective: download a Verilog design onto the Altera DE2 board and drive the seven-segment display.

1. Start a new project (activity5, top level counta) with the same device and EDA tool settings.
2. Create a new Verilog HDL file with the code in Listing 3.4; save as counta.
3. Compile and simulate the design.
4. Assign each output pin to a physical device pin using **Assignment** → **Pin Planner**, per Table 3.2 (refer to the DE2 user manual for pin locations).
5. Save, then recompile the project.
6. Download the configuration to the DE2 board via **Tools** → **Programmer** → **Hardware Setup** → USB-Blaster → **Start**.
7. Record the pattern displayed on the seven-segment display.
8. Change the assignment to assign `q[6:0] = 7'b1111000;`, save, recompile, and download again. Record the new displayed pattern.

```
counta.v
1 module counta(q);
2   output [6:0] q;
3   assign q[6:0] = 7'b0010000;
4 endmodule
```

Listing 3.4: counta.v – static seven-segment pattern.

Table 3.2: Seven-segment output pin assignment (DE2 board).

#	Node name	Direction	Pin
1	Q[6]	Output	
2	Q[5]	Output	
3	Q[4]	Output	
4	Q[3]	Output	
5	Q[2]	Output	
6	Q[1]	Output	
7	Q[0]	Output	

Activity 6 – 16-bit binary counter with frequency divider

Objective: design a 16-bit binary counter driven through a frequency divider.

1. Open Quartus II and write the frequency-divider module in Listing 3.5 (seven), which lowers the incoming clock frequency (Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N)).
2. Create a reusable symbol for seven.
3. Write the 4-bit counter module in Listing 3.6 (led).
4. Create a reusable symbol for led.
5. Draw the schematic in Figure 3.6, connecting the seven block's most significant output bit q[16] to the clock input of the led block.
6. Assign pin numbers as shown in Table 3.3 (refer to the DE2 user manual).
7. Use a function generator to supply the clock signal, using 1 MHz as the clock input.
8. Determine the effective input frequency reaching the led module.
9. Discuss your result based on the observed LED output display.

seven.v – frequency divider

```
1 module seven(q, clk);
2   output [15:0] q;
3   input clk;
4   reg [15:0] q;
5
6   always @(posedge clk)
7   begin
8     q = (q + 1) % 65536;
9   end
10 endmodule
```

Listing 3.5: seven.v – 16-bit frequency divider.

led.v – 4-bit counter

```
1 module led(x, clk);
2   input clk;
3   output [3:0] x;
4   reg [3:0] x;
5
6   always @(posedge clk)
7   begin
8     x = (x + 1) % 16;
9   end
10 endmodule
```

Listing 3.6: led.v – 4-bit binary counter.

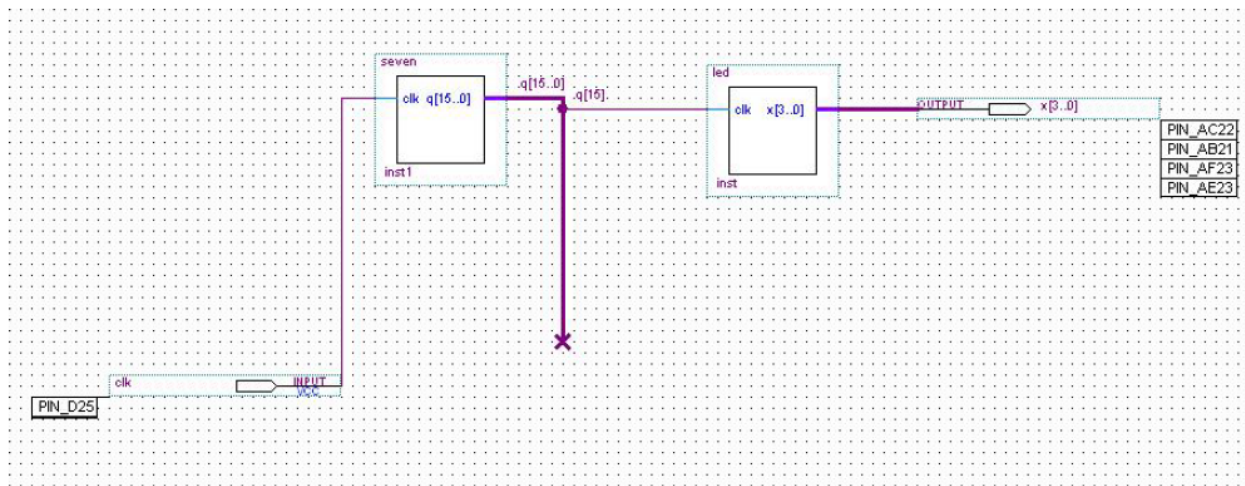


Figure 3.6: Schematic diagram for the 16-bit binary counter with frequency divider.

Table 3.3: Pin assignment for the 4-bit binary counter.

#	Node name	Direction	Pin location
1	clk	Input	
2	X[3]	Output	
3	X[2]	Output	
4	X[1]	Output	
5	X[0]	Output	

Activity 7 – 0–99 counter on seven-segment display

Objectives: (1) design a counter that counts 0 to 99; (2) display the result on the seven-segment displays.

1. Using a DIP switch as the trigger input, write Verilog code for a counter that counts from 0 to 99 once the switch is triggered.
2. Decode and display the result (tens and units digits) on two seven-segment displays.
3. Print and submit the Verilog code and simulation result.

Activity 8 – Running light

Objective: design a running-light display on the Altera DE2 board.

1. Using a DIP switch as the input, write Verilog code to implement a running-light (chasing LED) pattern.
2. Use three DIP switches to select between three different running-light patterns.
3. Print and submit the Verilog code and simulation result.



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 4

LCD INTERFACING TO ALTERA DE2 BOARD, AND STEPPER
MOTOR CONTROL WITH FPGA

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 4

LCD Interfacing to Altera DE2 Board, and Stepper Motor Control with FPGA

Objectives

1. To use Verilog code as a design entry method in Altera Quartus II Software.
2. To interface an LCD module to the Altera DE2 board.
3. To design digital circuits in an FPGA to control a stepper motor.
4. To simulate the circuit using Altera Quartus II Software.
5. To download the design onto an Altera DE2 board.

Equipment / Software Required

1. Altera Quartus II software
 - version 9/10/12 for Cyclone II
 - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Parallax 4-phase, 12 V, 75 Ω unipolar stepper motor
4. ULN2003 Darlington transistor array (motor driver)

Introduction

In recent years, LCDs have largely replaced simple LED displays: they can show numbers, characters, and graphics, and are relatively easy to program. Table 4.1 lists the LCD pins relevant to programming.

The *enable* (E) pin is used by the LCD to latch the information presented on its data pins: a high-to-low transition on E must be applied for the LCD to latch the data, and this pulse must be at least 450 ns wide. The required timing is shown in Figure 4.1.

Two important internal registers are selected via the RS pin. When RS = 0, the *instruction/command* register is selected, allowing commands to be sent to the LCD (Table 4.2 lists the available command codes). When RS = 1, the *data* register is selected, allowing ASCII character data to be written for display, sent through data pins D0–D7.

It is good practice to check the busy flag before writing to the LCD: with RS = 0 and $\overline{R/\overline{W}} = 1$, the busy flag appears on D7. While D7 = 1 the LCD is busy with internal operations and

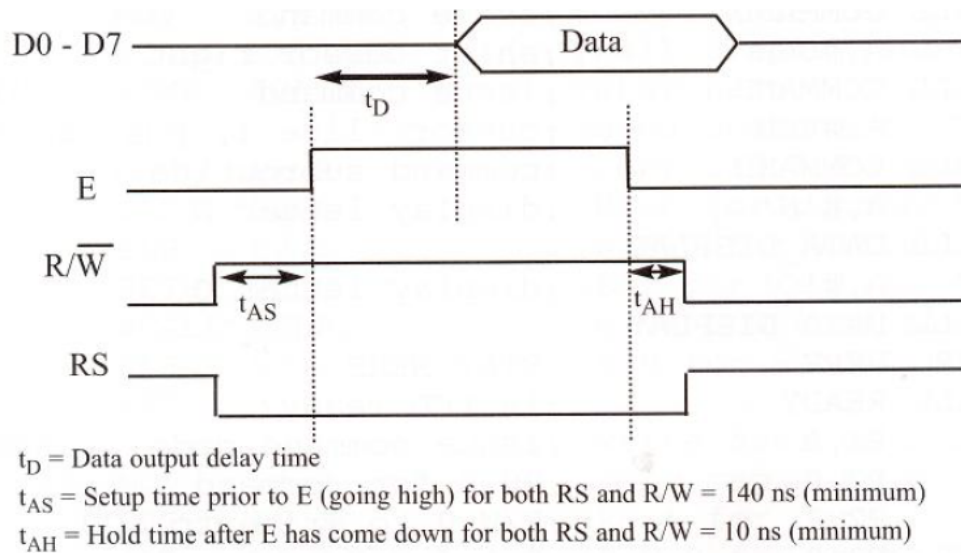
will not accept new data; once $D7 = 0$, the LCD is ready to receive the next byte.

Unlike an ordinary DC motor, a stepper motor does not spin freely when power is applied — it must be driven with a continuously pulsed sequence of specific coil-energising patterns. Each pulse advances the rotor by a fixed step angle (for this motor, 3.6° per step), which is what gives stepper motors their positioning precision: as long as speed and torque stay within the motor's limits, the controller always knows the rotor's exact position.

The rotor is driven by the interaction of magnetic fields as coils are switched on and off in sequence. Table 4.4 shows the normal four-coil unipolar step sequence; taking a step means driving two of the four coil-driver pins high at a time, in the order shown, through the ULN2003 driver.

Table 4.1: Pin description for the LCD module.

Pin	Symbol	I/O	Description
1	V_{ss}	—	Ground
2	V_{cc}	—	+5 V power supply
3	V_{ee}	—	Power supply to control contrast
4	RS	I	RS = 0: command register; RS = 1: data register
5	$R/\overline{W}$	I	0: write; 1: read
6	E	I/O	Enable (latches data on high-to-low transition)
7	DB0	I/O	8-bit data bus
8	DB1	I/O	
9	DB2	I/O	
10	DB3	I/O	
11	DB4	I/O	
12	DB5	I/O	
13	DB6	I/O	
14	DB7	I/O	



Note: Read requires an L-to-H pulse for the E pin.

Figure 4.1: LCD write-cycle timing diagram.

Table 4.2: LCD command codes.

Code (hex)	Command to LCD instruction register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor left)
6	Increment cursor (shift cursor right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position left
14	Shift cursor position right
18	Shift entire display left
1C	Shift entire display right
80	Force cursor to beginning of line 1
C0	Force cursor to beginning of line 2
38	2 lines, 5 × 7 dot matrix

Procedure

Activity 1 — Displaying lookup-table data on the LCD

Objective: display fixed text, stored in a lookup table, on the LCD.

1. Open Quartus II (Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N))
2. Write the Verilog modules given in Listing 4.1 (lcd_t) and Listing 4.2 (LCD_Controller) to send commands and data to the LCD.
3. lcd_t uses a lookup table (LUT) to store the sequence of commands and characters to send to the LCD.
4. Create a reusable symbol for each module, as in Experiment 3.
5. Draw the schematic in Figure 4.2, connecting lcd_t's outputs to LCD_Controller's inputs, and LCD_Controller's outputs to the LCD interface pins.
6. Assign pin numbers as given in the Altera DE2 board manual (Cyclone II or Cyclone IV).
7. Download the program to the Altera DE2 board.
8. Discuss the result based on your observation of the LCD.

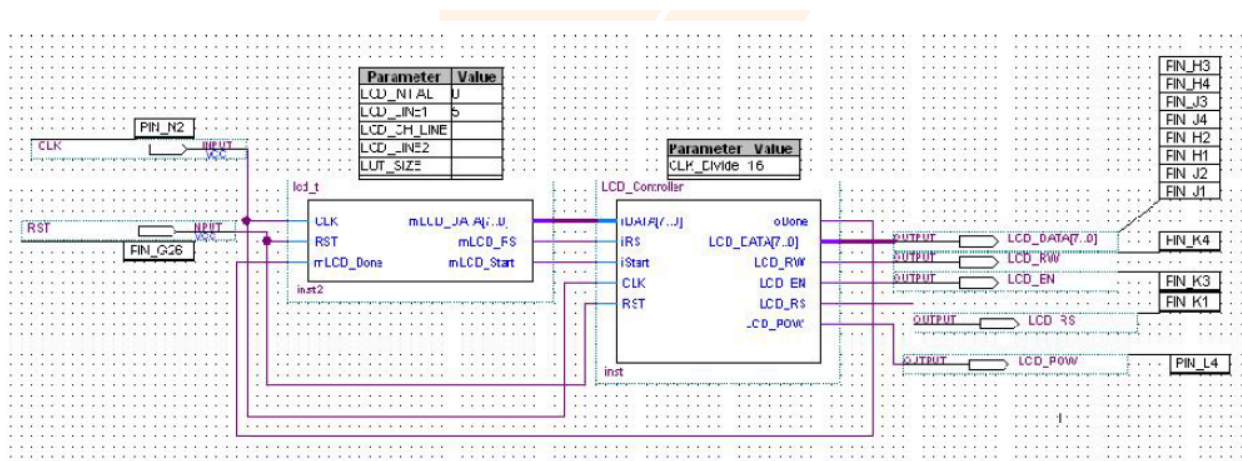


Figure 4.2: Schematic diagram for the LCD interface: lcd_t (lookup-table sequencer) feeding LCD_Controller (bus-timing controller).

Table 4.3: Pin assignment for the LCD interface.

#	Node name	Direction	Location
1	LCD_DATA[7]	Output	
2	LCD_DATA[6]	Output	
3	LCD_DATA[5]	Output	
4	LCD_DATA[4]	Output	
5	LCD_DATA[3]	Output	
6	LCD_DATA[2]	Output	
7	LCD_DATA[1]	Output	
8	LCD_DATA[0]	Output	
9	LCD_EN	Output	
10	LCD_POW	Output	
11	LCD_RS	Output	
12	LCD_RW	Output	
13	CLK	Input	
14	RST	Input	

lcd_t.v — Appendix 1: lookup-table sequencer (part 1 of 2)

```

1  module lcd_t (
2      // Host side
3      CLK, RST, mLCD_Done,
4      mLCD_DATA, mLCD_RS, mLCD_Start
5  );
6      // Host side
7      input  CLK, RST, mLCD_Done;
8      // Internal wires/registers
9      output [7:0] mLCD_DATA;
10     output mLCD_RS, mLCD_Start;
11     reg  [5:0] LUT_INDEX;
12     reg  [8:0] LUT_DATA;
13     reg  [5:0] mLCD_ST;
14     reg  [17:0] mDLY;
15     reg  mLCD_Start;
16     reg  [7:0] mLCD_DATA;
17     reg  mLCD_RS;
18     wire mLCD_Done;
19
20     parameter LCD_INITIAL = 0;
21     parameter LCD_LINE1   = 5;
22     parameter LCD_CH_LINE = LCD_LINE1 + 16;
23     parameter LCD_LINE2   = LCD_LINE1 + 16 + 1;
24     parameter LUT_SIZE    = LCD_LINE1 + 32 + 1;
25

```

```

26 // Sequencing state machine: send one LUT entry, wait for
27 // LCD_Controller to finish (mLCD_Done), pause, then advance.
28 always @(posedge CLK or negedge RST)
29 begin
30     if (!RST)
31     begin
32         LUT_INDEX  <= 0;
33         mLCD_ST     <= 0;
34         mDLY        <= 0;
35         mLCD_Start  <= 0;
36         mLCD_DATA   <= 0;
37         mLCD_RS     <= 0;
38     end
39     else if (LUT_INDEX < LUT_SIZE)
40     begin
41         case (mLCD_ST)
42         0: begin
43             mLCD_DATA <= LUT_DATA[7:0];
44             mLCD_RS   <= LUT_DATA[8];
45             mLCD_Start <= 1;
46             mLCD_ST   <= 1;
47         end
48         1: if (mLCD_Done) begin
49             mLCD_Start <= 0;
50             mLCD_ST   <= 2;
51         end
52     end

```

lcd_t.v – Appendix 1: lookup-table sequencer (part 2 of 2, continued)

```

1         2: if (mDLY < 18'h3FFFE)
2             mDLY <= mDLY + 1;
3         else begin
4             mDLY <= 0;
5             mLCD_ST <= 3;
6         end
7         3: begin
8             LUT_INDEX <= LUT_INDEX + 1;
9             mLCD_ST <= 0;
10        end
11    endcase
12    end
13    end
14    // Lookup table: LCD_INITIAL entries are setup commands
15    // (function set, display on, clear, entry mode, home);
16    // LCD_LINE1 / LCD_LINE2 entries are ASCII characters for
17    // each line, with the MSB (bit 8) marking RS = 1 (data).
18    always
19    begin
20        case (LUT_INDEX)
21        LCD_INITIAL+0: LUT_DATA <= 9'h038; // function set
22        LCD_INITIAL+1: LUT_DATA <= 9'h00C; // display on

```

```

23     LCD_INITIAL+2: LUT_DATA <= 9'h001; // clear display
24     LCD_INITIAL+3: LUT_DATA <= 9'h006; // entry mode set
25     LCD_INITIAL+4: LUT_DATA <= 9'h080; // cursor home, line 1
26     // Line 1: "Welcome to the "
27     LCD_LINE1+0:   LUT_DATA <= 9'h120;
28     LCD_LINE1+1:   LUT_DATA <= 9'h157;
29     LCD_LINE1+2:   LUT_DATA <= 9'h165;
30     // ... remaining characters follow the same pattern ...
31     LCD_LINE1+15:  LUT_DATA <= 9'h120;
32     LCD_CH_LINE:   LUT_DATA <= 9'h0C0; // move to line 2
33     // Line 2: "    EEM 441 Lab "
34     LCD_LINE2+0:   LUT_DATA <= 9'h120;
35     LCD_LINE2+4:   LUT_DATA <= 9'h145;
36     // ... remaining characters follow the same pattern ...
37     LCD_LINE2+15:  LUT_DATA <= 9'h120;
38     default:       LUT_DATA <= 9'h000;
39   endcase
40   end
41   endmodule
42

```

Listing 4.1: lcd_t.v — lookup-table LCD message sequencer.

LCD_Controller.v — Appendix 2: bus-timing controller (part 1 of 2)

```

1  module LCD_Controller (
2      // Host side
3      iDATA, iRS, iStart, oDone, CLK, RST,
4      // LCD interface
5      LCD_DATA, LCD_RW, LCD_EN, LCD_RS, LCD_POW
6  );
7      parameter CLK_Divide = 16;
8
9      // Host side
10     input  [7:0] iDATA;
11     input  iRS, iStart;
12     input  CLK, RST;
13     output reg oDone;
14
15     // LCD interface
16     output [7:0] LCD_DATA;
17     output reg LCD_EN;
18     output LCD_RW;
19     output LCD_RS;
20     output LCD_POW;
21
22     // Internal registers
23     reg [4:0] Cont;
24     reg [1:0] ST;
25     reg preStart, mStart;
26
27     // Write-only interface: bypass iRS straight to LCD_RS

```

```

28 assign LCD_DATA = iDATA;
29 assign LCD_RW    = 1'b0;
30 assign LCD_RS    = iRS;
31 assign LCD_POW   = 1'b1;
32 always @(posedge CLK or negedge RST)
33 begin
34     if (!RST)
35     begin
36         oDone    <= 1'b0;
37         LCD_EN    <= 1'b0;
38         preStart <= 1'b0;
39         mStart    <= 1'b0;
40         Cont      <= 0;
41         ST        <= 0;
42     end
43     else
44     begin
45         // Rising-edge detect on iStart
46         preStart <= iStart;
47         if ({preStart, iStart} == 2'b01)
48         begin
49             mStart <= 1'b1;
50             oDone  <= 1'b0;
51         end

```

LCD_Controller.v – Appendix 2: bus-timing controller (part 2 of 2, continued)

```

1     if (mStart)
2     begin
3         case (ST)
4             0: ST <= 1; // wait setup
5             1: begin
6                 LCD_EN <= 1'b1; // raise enable
7                 ST <= 2;
8             end
9             2: if (Cont < CLK_Divide) // hold enable pulse width
10                 Cont <= Cont + 1;
11             else
12                 ST <= 3;
13             3: begin
14                 LCD_EN <= 1'b0; // drop enable: latches data
15                 mStart <= 1'b0;
16                 oDone  <= 1'b1;
17                 Cont   <= 0;
18                 ST     <= 0;
19             end
20         endcase
21     end
22 end
23 end
24 endmodule

```

Listing 4.2: LCD_Controller.v – generates the LCD enable-pulse timing.

Activity 2 – Displaying DIP switch input on the LCD

Objective: display the live 8-bit DIP switch configuration on the LCD.

1. Write Verilog code that reads the 8-bit DIP switch configuration and displays it on the LCD (e.g. if switch 1 is turned on, the LCD should display 1 in the corresponding position).
2. Print and submit the Verilog code and simulation result.

Introduction

Unlike an ordinary DC motor, a stepper motor does not spin freely when power is applied – it must be driven with a continuously pulsed sequence of specific coil-energising patterns. Each pulse advances the rotor by a fixed step angle (for this motor, 3.6° per step), which is what gives stepper motors their positioning precision: as long as speed and torque stay within the motor's limits, the controller always knows the rotor's exact position.

The rotor is driven by the interaction of magnetic fields as coils are switched on and off in sequence. Table 4.4 shows the normal four-coil unipolar step sequence; taking a step means driving two of the four coil-driver pins high at a time, in the order shown, through the ULN2003 driver.

Table 4.4: Normal step sequence for a four-coil unipolar stepper motor.

Coil	Step 1	Step 2	Step 3	Step 4	Step 1
1 (B)	1	1	0	0	1
2 ($\bar{B}$)	0	0	1	1	0
3 (A)	1	0	0	1	1
4 ($\bar{A}$)	0	1	1	0	0

Activity 3 – Verilog stepper motor controller

Objective: write Verilog code to control the motion of a stepper motor.

1. Open Quartus II and start the New Project Wizard.
 - Working directory: fpga; project name: activity1; top-level entity: step.
 - Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N).
 - Accept the default EDA tool settings and finish.
2. Create a new Verilog HDL file and enter the code in Listing 4.3 (step, divider, and stepper modules).
3. Save as step and start compilation.
4. Use **Tools** → **Netlist Viewer** → **RTL Viewer** to generate the block overview shown in Figure 4.3.

- Assign pin numbers as shown in Table 4.5 (refer to the DE2 user manual). For the phaseout outputs that interface with the stepper motor driver, use expansion header GPIO_0.
- Download the program to the Altera DE2 board.
- Discuss the result.

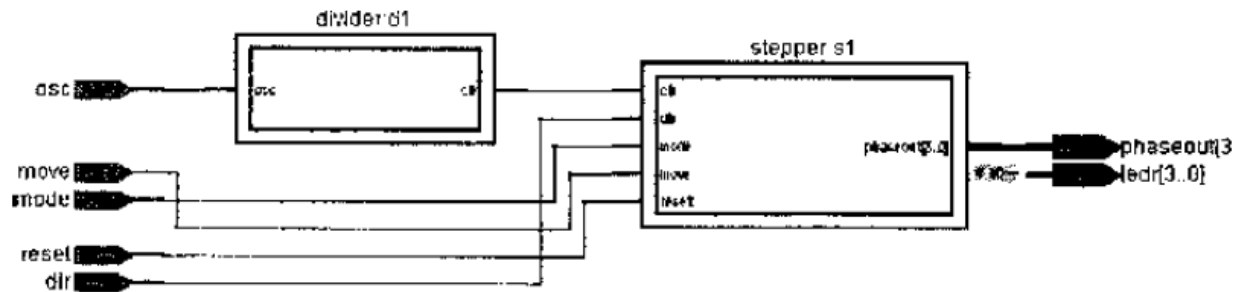


Figure 4.3: RTL Viewer overview of the step top-level module.

Table 4.5: Pin assignment for stepper motor control.

#	Node name	Direction	Pin location	#	Node name	Direction	Pin location
1	osc	Input		5	ledr[0]	Output	
	dir (SW0)	Input		6	phaseout[3]	Output	
	mode (SW1)	Input		7	phaseout[2]	Output	
	move (SW2)	Input		8	phaseout[1]	Output	
	reset (SW3)	Input		9	phaseout[0]	Output	
2	ledr[3]	Output					
3	ledr[2]	Output					
4	ledr[1]	Output					

step.v – top-level module

```

1  //=====
2  // Stepper Motor Controller
3  //=====
4  // TOP LEVEL MODULE
5  // reset (active low) brings the shaft to its reference position.
6  // mode selects step (1) or continuous (0) operation.
7  // dir selects clockwise (1) or anti-clockwise (0) motion.
8  // move shifts the shaft by one half-step in step mode.
9  module step(osc, reset, dir, mode, move, phaseout, ledr);
10     input  osc, reset, mode, dir, move;
11     output [3:0] phaseout, ledr;
12
13     wire clk;
14
15     divider d1 (
16         .osc (osc),
17         .clk (clk)
18     );
19
20     stepper s1 (
21         .clk      (clk),
22         .reset     (reset),
23         .dir       (dir),
24         .mode      (mode),
25         .move      (move),
26         .phaseout  (phaseout)
27     );
28 endmodule

```

step.v – motor controller module (part 1 of 2)

```

1  // Module stepper is the motor controller.
2  module stepper(clk, reset, dir, mode, move, phaseout);
3     input  clk, reset, dir, mode, move;
4     output [3:0] phaseout;
5     reg [3:0] phaseout, position, ledr;
6
7     always @(posedge clk or negedge reset)
8     begin
9         if (reset == 0) begin
10             // motor resets to phase 1000 when reset is asserted
11             phaseout = 4'b1000;
12             ledr      = phaseout;
13         end
14         else begin
15             if (mode == 0) begin
16                 // Mode = 0: continuous motion.
17                 // The 8-state sequence below follows Table 1 in both
18                 // directions; only the first two states are shown here,
19                 // the remaining six states (0100, 0110, 0010, 0011,
20                 // 0001, 1001) follow the identical if(dir==0)/else

```

```

21 // pattern, cycling back to 1000.
22 case (phaseout)
23     4'b1000: if (dir == 0) begin
24         phaseout = 4'b1100; ledr = phaseout;
25     end else begin
26         phaseout = 4'b1001; ledr = phaseout;
27     end
28     4'b1100: if (dir == 0) begin
29         phaseout = 4'b0100; ledr = phaseout;
30     end else begin
31         phaseout = 4'b1000; ledr = phaseout;
32     end
33     // ... 4'b0100, 4'b0110, 4'b0010, 4'b0011,
34     //      4'b0001, 4'b1001 follow the same pattern ...
35     default: $display("invalid input");
36 endcase
37 end
38 else begin
39     // Mode = 1: single-step motion, advances only when
40     // move == 0. Uses the same 8-state sequence as above,
41     // indexed by `position` instead of `phaseout`.
42     if (move == 1'b0) begin
43         case (position)
44             // ... identical 8-state sequence as continuous
45             // mode, see Table 1 ...
46             default: $display("invalid input");
47         endcase
48     end
49 end
50 end
51 end

```

step.v — motor controller module (part 2 of 2)

```

1 // Motor advances by one half-step on the falling edge of move.
2 always @(negedge move or negedge reset)
3 begin
4     if (reset == 0) begin
5         position = 4'b1000; // NOTE: corrected from the original
6                             // listing, which printed this value
7                             // as "4.101011" -- likely a
8                             // transcription error; 4'b1000
9                             // matches the reset state used
10                            // consistently elsewhere.
11         ledr = position;
12     end
13     else if (reset == 1) begin
14         position = phaseout;
15         ledr = phaseout;
16     end
17 end
18 endmodule

```

step.v – clock divider module

```

1 // Module divider divides the incoming clock to produce a
2 // clock suitable for driving the motor controller.
3 module divider(osc, clk);
4     input  osc;
5     output clk;
6     reg    clk;
7     reg [15:0] count;
8
9     initial count = 16'b0000000000000000;
10
11    always @(posedge osc)
12    begin
13        count = count + 1;
14        clk    = count[15];
15    end
16 endmodule

```

Listing 4.3: step.v – top-level, motor-controller, and clock-divider modules.

Activity 4 – Variable-speed control with display feedback

Objective: extend the controller with selectable speed and direction/mode feedback.

1. Write Verilog code that allows selection of three different speeds for the stepper motor in continuous-motion mode.
2. Display the motor's direction of rotation on the seven-segment display, and the current mode (step / continuous) on the LCD display.
3. Print and submit the Verilog code and schematic diagram.

Appendix: Stepper Motor Datasheet Reference

Relevant specifications and wiring information for the Parallax #27964 stepper motor used in this experiment, extracted from the manufacturer datasheet (see References).



Figure 4.4: Parallax #27964 stepper motor.

Table 4.6: Technical specifications, Parallax #27964 stepper motor.

Parameter	Value
Type	4-phase, 12 V unipolar
Step angle	3.6° per step
Phase resistance	75 Ω
Current	150 mA
Phase inductance	39 mH
Detent torque	80 g·cm
Holding torque	600 g·cm
Mounting hole spacing (diagonal)	1.73 in
Mounting hole diameter	0.11 in
Shaft diameter	0.197 in
Shaft length	0.43 in
Motor diameter	1.66 in
Motor height	1.35 in
Weight	0.55 lb

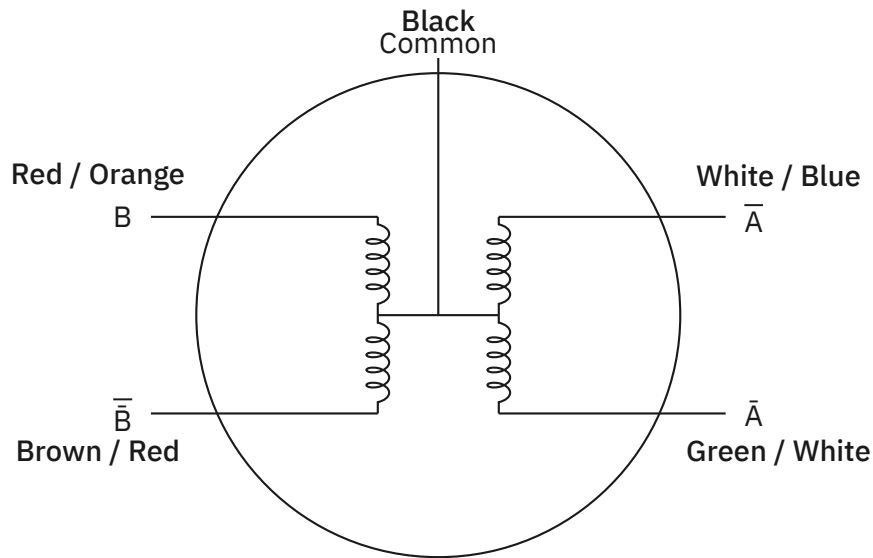


Figure 4.5: Coil winding and lead-wire colours, Parallax #27964 stepper motor.

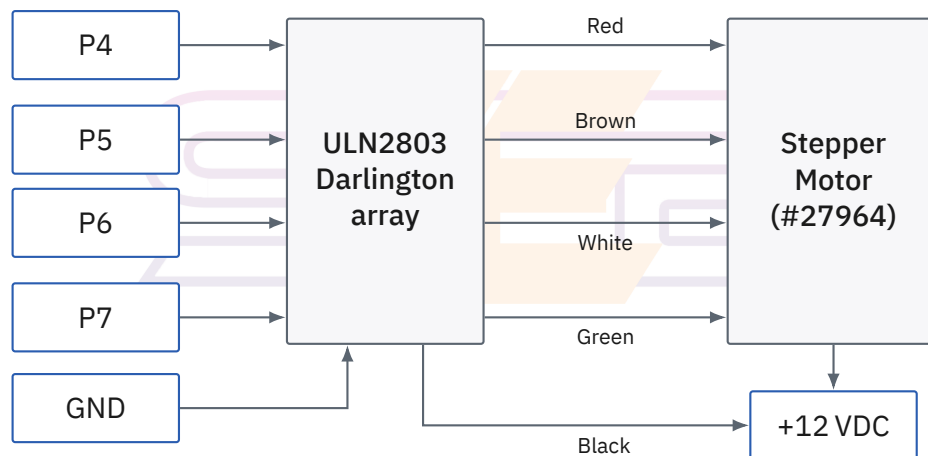


Figure 4.6: Reference driver-wiring diagram.

Note:

1. The P4–P7 labelling above follows the datasheet’s original microcontroller example; in this experiment, the corresponding driver inputs are instead the phaseout [3:0] outputs of the stepper module, assigned to DE2 GPIO pins as listed in Table 4.5.
2. The GND are to be common ground with the DE2 board, ULN2803 and the power source.
3. The +12 VDC is to be supplied from an external power source, not the DE2 board.

References

1. Altera DE2 User Manual.
2. *Introduction to Quartus II Software – Verilog, VHDL, Schematic*, Altera University Program.

3. *Introduction to Verilog*, Carleton University course notes.
4. *Stepper Motor Controller Using MAX II CPLDs*, Altera Application Note AN-488.
5. Parallax Inc., *4-Phase 12-Volt 75 Ω Unipolar Stepper Motor (#27964)*, technical datasheet.



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 5

**INTERFACING FPGA TO AN ANALOG-TO-DIGITAL
CONVERTER**

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 5

Interfacing FPGA to Analog-to-Digital Converter

Objectives

1. To understand the operation of an analog-to-digital converter (ADC).
2. To interface an FPGA, via the Altera DE2 board, to an ADC.

Equipment / Software Required

1. Altera Quartus II software
 - version 9/10/12 for Cyclone II
 - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Multimeter / oscilloscope
4. ADC0804 analog-to-digital converter
5. Resistors: 10 k Ω
6. Variable resistor (potentiometer), 10 k Ω
7. Capacitor: 150 pF
8. 8 LEDs
9. 74LS245 octal bus transceiver

Introduction

This experiment explores interfacing an ADC chip to an FPGA. ADCs are among the most widely used data-acquisition devices: they translate an analog signal into a digital number that a processor (or, here, an FPGA) can read. A widely used ADC is the ADC0804 (National Semiconductor): it operates from +5 V and has 8-bit resolution.

Besides resolution, *conversion time* is a major factor in judging an ADC: the time the ADC takes to convert its analog input to a digital number. For the ADC0804 this depends on the clock signal applied to the CLK R / CLK IN pins, but cannot be faster than 110 μ s.

The ADC0804's input range is set by the $V_{ref}/2$ pin, as shown in Table 5.1. The chip's outputs are 5 V logic; since the DE2 board's inputs only tolerate 3.3 V, an octal bus transceiver (74LS245) must be used to step the output voltage down before it reaches the FPGA.

$$D_{out} = \frac{V_{in}}{\text{step size}}$$

where D_{out} is the digital data output and V_{in} is the analog input voltage.

Data conversion with the ADC0804 follows this sequence (Figure 5.1):

- Set $\overline{CS} = 0$ and send a low-to-high pulse on the $\overline{WR}$ pin to start the conversion.
- Monitor the $\overline{INTR}$ pin: while it is high, the conversion is still in progress; once it goes low, the conversion is complete.
- With $\overline{INTR}$ low, set $\overline{CS} = 0$ again and send a high-to-low pulse on the $\overline{RD}$ pin to read the data out of the ADC0804.

Table 5.1: $V_{ref}/2$ relation to V_{in} range.

$V_{ref}/2$ (V)	V_{in} range (V)	Step size (mV)
Not connected	0 to 5	$5/256 = 19.53$
2.0	0 to 4	$4/255 = 15.62$
1.5	0 to 3	$3/256 = 11.71$
1.28	0 to 2.56	$2.56/256 = 10.00$
1.0	0 to 2	$2/256 = 7.81$
0.5	0 to 1	$1/256 = 3.90$

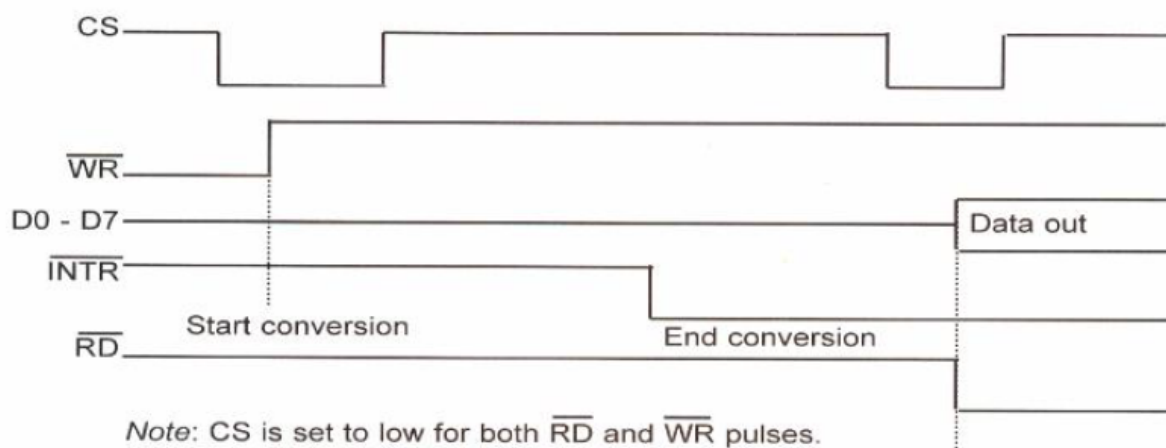


Figure 5.1: Read and write timing for the ADC0804.

Procedure

Activity 1 – Free-running mode

Objective: run the ADC0804 in free-running mode.

- Connect the ADC0804 as shown in Figure 5.2 for free-running mode.

2. Vary the input signal from 0 to 5 V using the 10 k Ω potentiometer shown in Figure 5.2 as the ADC's input. The input must not exceed 5 V.
3. Tabulate your results in Table 5.2.
4. Compare your hardware result with the value calculated using Equation (1).

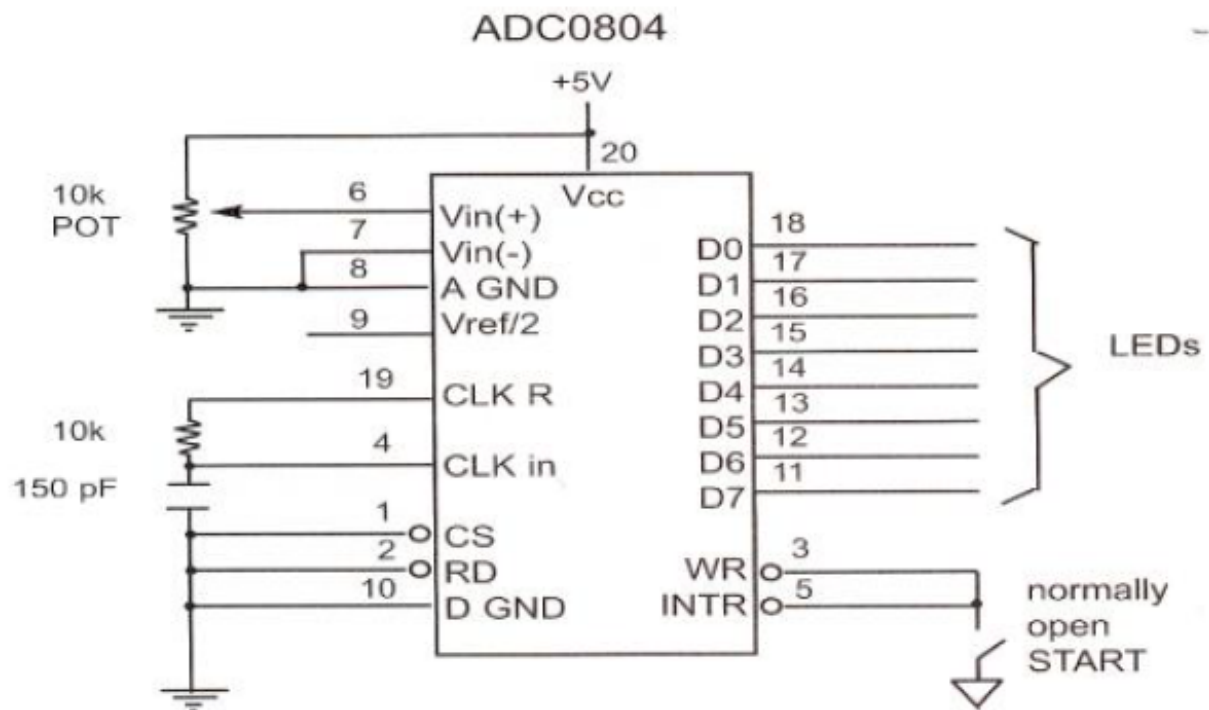


Figure 5.2: ADC0804 test circuit for free-running mode.

Table 5.2: Free-running mode output.

#	Input (V)	Output (hardware)	Output (calculation)
1	0.0		
2	0.5		
3	1.0		
4	1.5		
5	2.0		
6	2.5		
7	3.0		
8	3.5		
9	4.0		
10	4.5		

Activity 2 – Interfacing the ADC to the FPGA, displaying on LEDs

Objective: interface the ADC to the FPGA and display the digitised result on LEDs.

1. Use the circuit from Figure 5.2, modified as follows.
2. Connect the $\overline{RD}$ and $\overline{WR}$ pins to general-purpose I/O pins on the DE2 board, declared as **outputs**.
3. Connect the $\overline{INTR}$ and D0–D7 pins to the inputs of a 74LS245. Connect the 74LS245's outputs to general-purpose I/O pins on the DE2 board, declared as **inputs**. Refer to the DE2 user manual for pin locations.
4. In Verilog, generate the $\overline{WR}$ pulse to start conversion, poll $\overline{INTR}$ for end of conversion, then pulse $\overline{RD}$ to read the data on D0–D7.
5. Assign the FPGA's digitised outputs to the LED pins.
6. Compile, simulate, and download the program to the DE2 board.
7. Use the potentiometer to vary the input value.
8. Tabulate your results in Table 5.3.
9. Print and submit the Verilog code, simulation result, and experimental results.

Table 5.3: FPGA-interfaced ADC output.

#	Input (V)	Output
1	0.8	
2	1.7	
3	2.3	
4	2.8	
5	3.1	
6	3.6	
7	3.9	
8	4.1	
9	4.6	
10	4.8	

Activity 3 – Displaying the ADC result on the LCD

Objective: interface the ADC0804 to the DE2 board and display the result on the LCD.

1. Using the same circuit as in Activity 2, write Verilog code to read data from the ADC and display the result on the LCD (see Experiment 5 for the LCD interface).
2. Print and submit the Verilog code and simulation result.



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 6

INTERFACING FPGA TO A DIGITAL-TO-ANALOG CONVERTER

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 6

Interfacing FPGA to Digital-to-Analog Converter

Objectives

1. To understand the operation of a digital-to-analog converter (DAC).
2. To interface an FPGA, via the Altera DE2 board, to a DAC.

Equipment / Software Required

1. Altera Quartus II software
 - version 9/10/12 for Cyclone II
 - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Multimeter / oscilloscope
4. DAC0800 digital-to-analog converter
5. Resistors: 10 k Ω ($\times 4$), 100 Ω ($\times 1$)
6. Capacitor: 0.01 μF

Introduction

A digital-to-analog converter (DAC) converts digital pulses into an analog signal. The first criterion for judging a DAC is its resolution, which depends on the number of binary inputs: the number of distinct analog output levels equals 2^n , where n is the number of data-bit inputs. An 8-bit DAC such as the DAC0800 therefore provides 256 discrete voltage (or current) output levels.

A DAC can be used to generate a sine wave. The output voltage for a given angle θ is:

$$V_{out} = 5\text{ V} + (5\text{ V} \times \sin \theta)$$

To find the digital value to send to the DAC for a given angle, multiply the resulting V_{out} by 25.6 (since there are 256 steps across a 10 V full-scale output).

Procedure

Activity 1 – DAC0800 operation

Objective: understand the operation of the DAC0800.

1. Connect the DAC0800 as shown in Figure 6.1.

2. Apply the digital input values at D0–D7 as listed in Table 6.1.
3. Measure the resulting analog output and tabulate your results in Table ??.

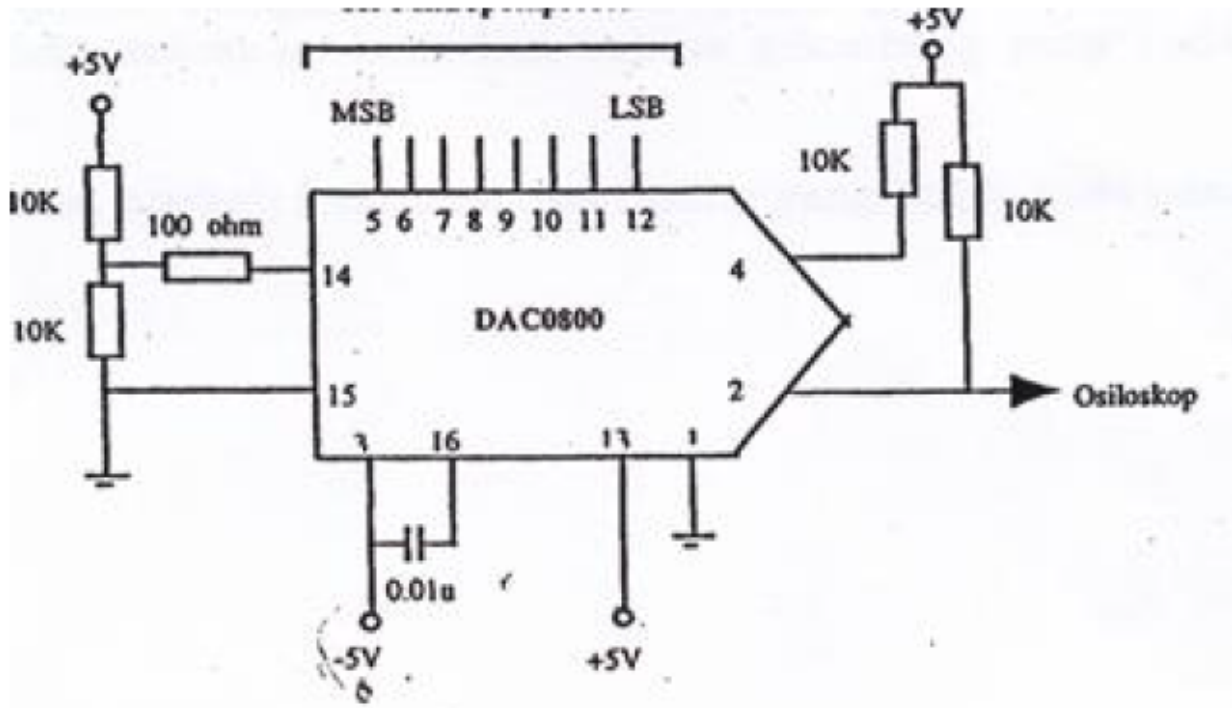


Figure 6.1: DAC0800 test circuit with op-amp output stage.

Table 6.1: DAC0800 digital-to-analog conversion results.

Digital value (decimal)	Analog value
0	
10	
60	
80	
100	
128	
180	
200	
220	
255	

Activity 2 – Generating a square wave

Objective: interface the FPGA to the DAC0800 and generate a square wave.

1. Connect the circuit as shown in Figure ??.
2. Connect D0–D7 of the DAC to general-purpose I/O pins on the DE2 board.
3. Write Verilog code to generate a square wave with a frequency of 100 Hz and a 70% duty cycle.
4. You may use a lookup table, following the same approach as the LCD program in Experiment 5.
5. Print and submit the Verilog code and simulation result.

Activity 3 – Generating a sine wave

Objective: interface the FPGA to the DAC0800 and generate a sine wave.

1. Using the equation in the Introduction, complete Table 6.2.
2. Connect the circuit as shown in Figure ??.
3. Connect D0–D7 of the DAC to general-purpose I/O pins on the DE2 board.
4. Write Verilog code to generate a sine wave with a frequency of 100 Hz.
5. You may use a lookup table, following the same approach as the LCD program in Experiment 5.
6. Print and submit the Verilog code and simulation result.

Table 6.2: Angle vs. voltage magnitude for the sine wave.

Angle (°)	$\sin \theta$	V_{out} (V)	Value sent to DAC ($V_{out} \times 25.6$)
0			
30			
60			
90			
120			
150			
180			
210			
240			
270			
300			
330			
360			



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 7

DC MOTOR SPEED MEASUREMENT

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 7

DC Motor Speed Measurement System

Objectives

1. To understand the methods of acquiring measurements automatically with the aid of a computer.
2. To become familiar with the data-acquisition (DAQ) environment.
3. To become familiar with signal conditioning in a computer-based data-acquisition system.
4. To design and build a signal-conditioning circuit whose parameters are justified by a measurement, rather than assumed.
5. To implement a computer-based measurement system for the rotational speed of a DC motor.

Equipment / Software Required

- DC motor trainer with a 12-slot wheel encoder
- Digital storage oscilloscope (DSO)
- Components for the student-designed filter and signal-conditioning circuit (op-amps, resistors, capacitors)
- ADAM-3968 wiring terminal board
- PCIE-1816 multifunction DAQ card, installed in the host PC
- PC with a C/C++ or Python development environment

Introduction

In most measurement processes it is often necessary to acquire data automatically, without a human operator. A computer-based instrument has several advantages over conventional measuring instruments: it can acquire measurements quickly and accurately, sustain real-time control, and operate for extended periods with consistent precision.

A computer-based measurement system typically contains the following elements (Figure 7.1): a transducer, a signal-conditioning stage, data-acquisition hardware, analysis hardware, and software running on the host PC.

Signal conditioning means manipulating an analogue signal so that it meets the requirements of the next processing stage — most commonly, so that it is suitable for an analogue-to-digital converter. Signal conditioning can include amplification, filtering, range matching, and isolation.

Filtering is the most common signal-conditioning function: frequencies that do not carry valid data are attenuated. In this experiment the encoder produces a pulse train whose useful frequency content is bounded, and any filter used ahead of the DAQ card should be designed — not assumed — around that bound.

Amplification increases the resolution of the input signal and improves its signal-to-noise ratio.

Isolation allows a signal to pass from its source to the measurement device without a direct physical (galvanic) connection, protecting sensitive equipment from the signal source and vice versa.

The Sallen–Key topology is a widely used active-filter structure, valued for its simplicity: it realises a two-pole (12 dB/octave) low-pass, high-pass, or band-pass response using a single op-amp configured as a near-unity-gain voltage-controlled voltage source (VCVS), together with two resistors and two capacitors that set the cutoff frequency and Q .

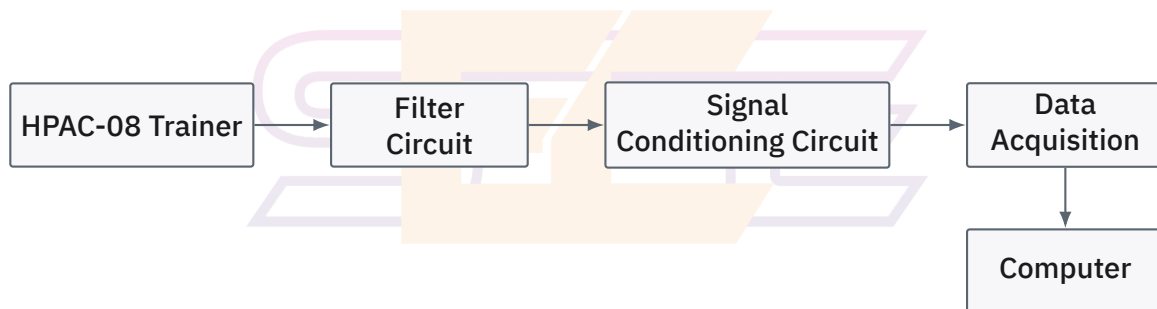


Figure 7.1: Computer-based measurement system: transducer, signal conditioning, DAQ hardware, and analysis software.

Procedure

Activity 1 — Characterising the encoder signal and designing the filter

Objective: measure the encoder's pulse characteristics and design a signal-conditioning filter around the measured cutoff frequency, rather than an assumed one.

1. Wire the encoder's output test point to the oscilloscope.
2. Run the motor across its usable speed range and observe the encoder pulse train on the oscilloscope.
3. At a representative operating speed, measure the pulse period and estimate the corresponding fundamental frequency from the oscilloscope trace.
4. From this measurement, determine a suitable cutoff frequency for noise-rejection filter.

- Design the necessary filter based on the cutoff frequency determined prior. Justify your design choices.
- Design the signal-conditioning circuit needed to present the filtered encoder pulses to the PCIE-1816's digital input with valid logic levels. Justify your design choices.

Activity 2 – Interfacing to the DAQ hardware

Objective: connect the conditioned encoder signal to the PC via the ADAM-3968 and PCIE-1816.

- Connect the output of your signal-conditioning circuit to a digital input terminal on the ADAM-3968 terminal board.
- Verify the voltage needed by PCIE1816 for it to detect HIGH/LOW logic using the NAVIGATOR application in the PC before writing any code, to confirm the expected pulses are being registered.



Figure 7.2: Signal path from the DC motor trainer to the host PC.

Activity 3 – Speed measurement software

Objective: implement software to compute and display motor speed (frequency and RPM) in real time, using two selectable measurement methods.

Two complementary methods can be used to convert the pulse train into a frequency and rotational speed, and your software should let the user select between them:

Method 1 – by counts (fixed pulse count, 1–99): the user specifies a target pulse count N ($1 \leq N \leq 99$); the software measures the elapsed time T_N for N pulses to occur, then computes

$$f = \frac{N}{T_N \times 12} \text{ [rev/s]}, \quad \text{RPM} = 60f, \quad \omega = 2\pi f \text{ [rad/s]}$$

This method gives better resolution at high speed (many pulses arrive quickly, so T_N is measured over a longer, more precise interval) but responds more slowly at low speed, since a full count of N pulses can take a long time.

Method 2 – by period (fixed period window, 1–9 s): the user specifies a measurement window T seconds ($1 \leq T \leq 9$); the software counts the number of pulses N_T that occur within that window, then computes

$$f = \frac{N_T}{T \times 12} \text{ [rev/s]}, \quad \text{RPM} = 60f, \quad \omega = 2\pi f \text{ [rad/s]}$$

In both methods, t_{start} and t_{stop} (marking the beginning and end of the measurement) are obtained from the DAQ card's counter/timer using

$$\text{Elapsed time} = \frac{t_{\text{stop}} - t_{\text{start}}}{\text{CLK_TCK}}$$

where CLK_TCK is the counter's clock tick rate.

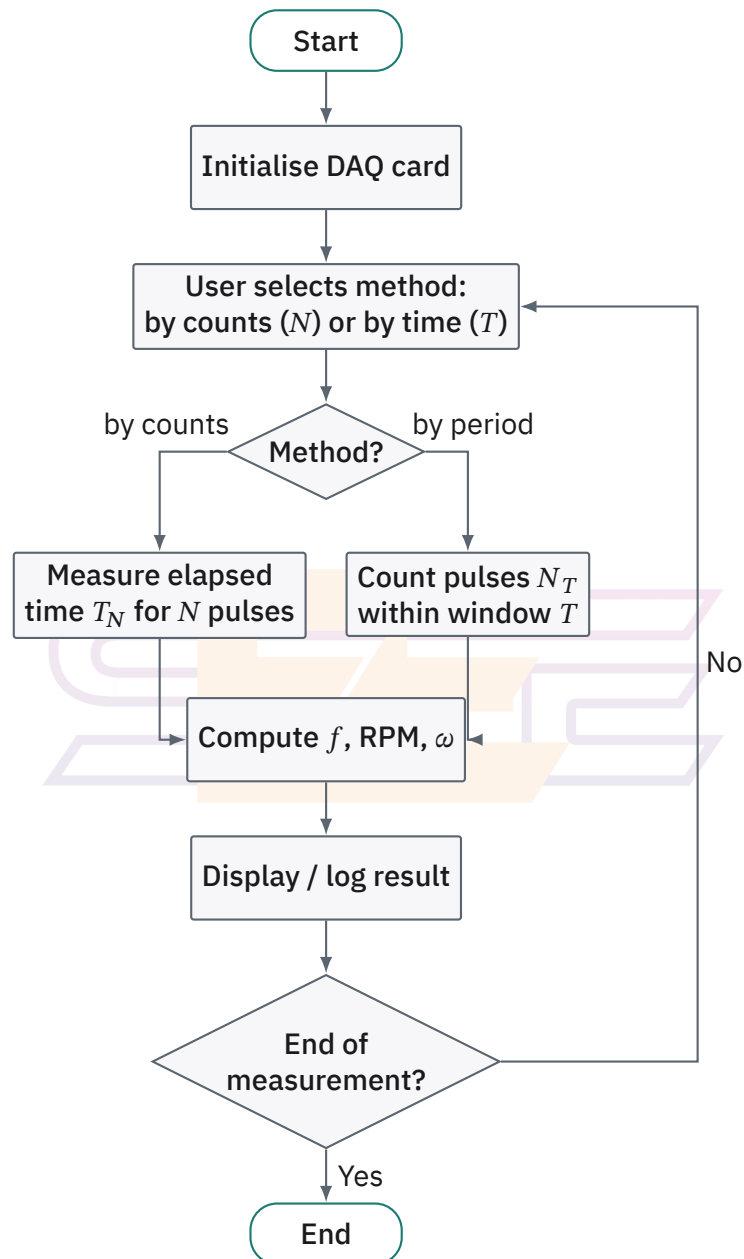


Figure 7.3: Measurement loop for the speed-acquisition software

1. In C, C++, or Python, write a program that:

- initialises the PCIE-1816 (via the vendor driver / SDK, or the DAQ card's counter API),
- lets the user select the measurement method (by counts, $1 \leq N \leq 99$; or by time, $1 \leq T \leq 9$ s) and enter the corresponding parameter,

- for the *by counts* method: times how long it takes for N encoder pulses to occur,
 - for the *by time* method: counts how many encoder pulses occur within the window T ,
 - computes the frequency, RPM, and angular speed ω using the relations above, and
 - displays the running value (frequency and RPM, with the active method indicated) in a simple GUI, updating in real time.
2. Interface your filter/conditioning circuit, the ADAM-3968, and the PCIE-1816 together and perform measurements across a range of motor input voltages, using both methods at each operating point. Compare your computed speed against a direct oscilloscope measurement of the encoder period at the same operating point.
 3. Tabulate your results (Tables 7.2–7.3).
 4. Plot computed speed (from both software methods) against the oscilloscope-measured speed.
 5. Discuss and comment on your results, including any discrepancy between the two software methods and the oscilloscope reference, and how each method's resolution changes with motor speed (see the trade-off noted above).

Table 7.1: By-counts method: frequency comparison table.

N	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Table 7.2: By-counts method — RPM comparison table.

N	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Table 7.3: By-period method: frequency comparison table.

T (s)	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Table 7.4: By-period method: RPM comparison table.

T (s)	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Report

1. Present your oscilloscope measurements of the encoder pulse and your resulting filter design (topology, calculations, and component values), together with the measured response of the built filter.
2. Present your design of the signal-conditioning stage between the filter and the PCIE-1816 digital input, with justification.
3. Compare your software-computed speed (both by-counts and by-time methods) with the oscilloscope-measured speed across your tested operating points, and discuss the sources of any discrepancy.
4. Compare the by-counts and by-time methods directly: at what motor speeds does each method give better resolution or a more stable reading, and why? Which would you recommend for a low-speed application, and which for a high-speed one?
5. What is the theoretical maximum measurable speed of this system, given the encoder resolution (12 slots/revolution) and your software's timing resolution?



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 8

TEMPERATURE MEASUREMENT AND CONTROL

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 8

Temperature Control System

Objectives

1. To design and build a signal-conditioning circuit for an NTC thermistor.
2. To interface the conditioned temperature signal to the PC via the PCIE-1750-U and ADAM-3937.
3. To implement a computer-based system that displays the measured temperature and performs ON/OFF control of an indicator LED on the trainer.

Equipment / Software Required

- NTC thermistor and trainer board with indicator LED
- Components for the student-designed signal-conditioning circuit (op-amps, resistors, capacitors)
- ADAM-3937 DB37 wiring terminal board
- PCIE-1750-U isolated digital I/O card, installed in the host PC
- PC with a C/C++ or Python development environment
- Digital multimeter

Introduction

In many industrial processes, temperature must be regulated automatically. Two general strategies are used: *logic (ON/OFF) control*, in which a heater, cooler, or indicator is simply switched on or off when a threshold is crossed, and *fuzzy/proportional control*, which requires a more elaborate definition of “hot” or “cold” before acting. This experiment uses logic control.

An NTC (negative temperature coefficient) thermistor is a resistor whose resistance decreases as its temperature increases. Given the thermistor’s resistance–temperature characteristic (from its datasheet), a measured resistance can be converted to a temperature, and vice versa.

The signal path for this experiment is: the thermistor’s resistance is converted to a voltage by a signal-conditioning circuit (which the student must design), this voltage is digitised and read by the PC via the PCIE-1750-U (through the ADAM-3937 wiring terminal), and software on the PC computes the corresponding temperature, displays it, and drives the indicator LED on the trainer according to a threshold comparison (Figure 8.1).

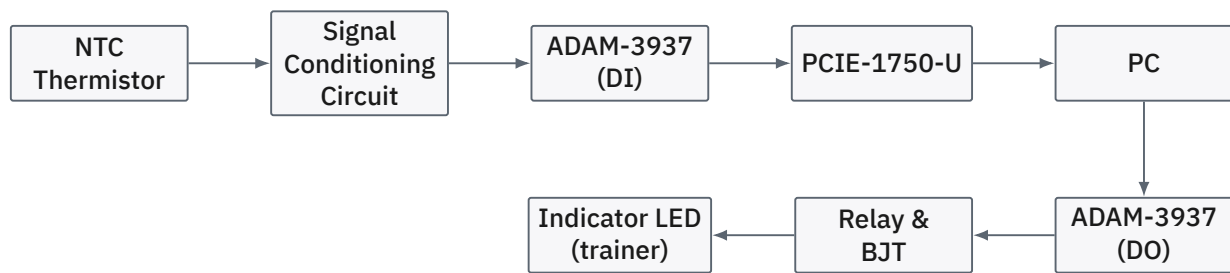


Figure 8.1: Block diagram of the temperature measurement and control system.

Procedure

Activity 1 – Signal-conditioning circuit design

Objective: design and build a circuit that converts the thermistor's resistance to a voltage suitable for the PCIE-1750-U input.

1. Obtain the resistance–temperature characteristic for the NTC thermistor from its datasheet, and determine the resistance range corresponding to the temperature range you intend to measure and control.
2. Design a signal-conditioning circuit that converts the thermistor's resistance to a voltage. Calculate your component values and justify your design choices, including the expected output voltage range over your chosen temperature range.
3. Include a voltage-follower (buffer) stage to eliminate loading effects between the conditioning circuit and the downstream measurement hardware.
4. Build the circuit and verify its output voltage against a reference temperature measurement at two or more points, and check linearity/consistency with your design calculations.

Activity 2 – Interfacing to the DAQ hardware

Objective: connect the conditioned temperature signal and the LED control line to the PC via the ADAM-3937 and PCIE-1750-U.

1. Connect the output of your signal-conditioning circuit to an input channel on the ADAM-3937 terminal board.
2. Connect the trainer's indicator LED control line to an isolated digital output channel on the ADAM-3937 via BJT/Relay circuit.
3. Connect the ADAM-3937 to the PCIE-1750-U card installed in the host PC via the supplied cable.
4. Verify both the input and output wiring using the DAQ card's bundled configuration/monitoring utility before writing any code: confirm the conditioned voltage reads correctly, and confirm the LED can be toggled from the utility.

Activity 3 – Temperature monitoring and LED control software

Objective: implement software that displays the measured temperature and controls the LED according to a threshold.

1. In C, C++, or Python, write a program that:
 - Periodically reads the conditioned thermistor voltage,
 - Converts the reading to a temperature using your circuit's calibration from Activity 1,
 - Displays the current temperature in a simple GUI, updating in real time, and
 - compares the measured temperature against the threshold and sets the LED outputs accordingly.
2. Report and justify your design choice for signal conditioning circuit.
3. Use the flowchart in Figure 8.2 as a starting point for your control logic, adapting the ON/OFF branch to match your chosen design.

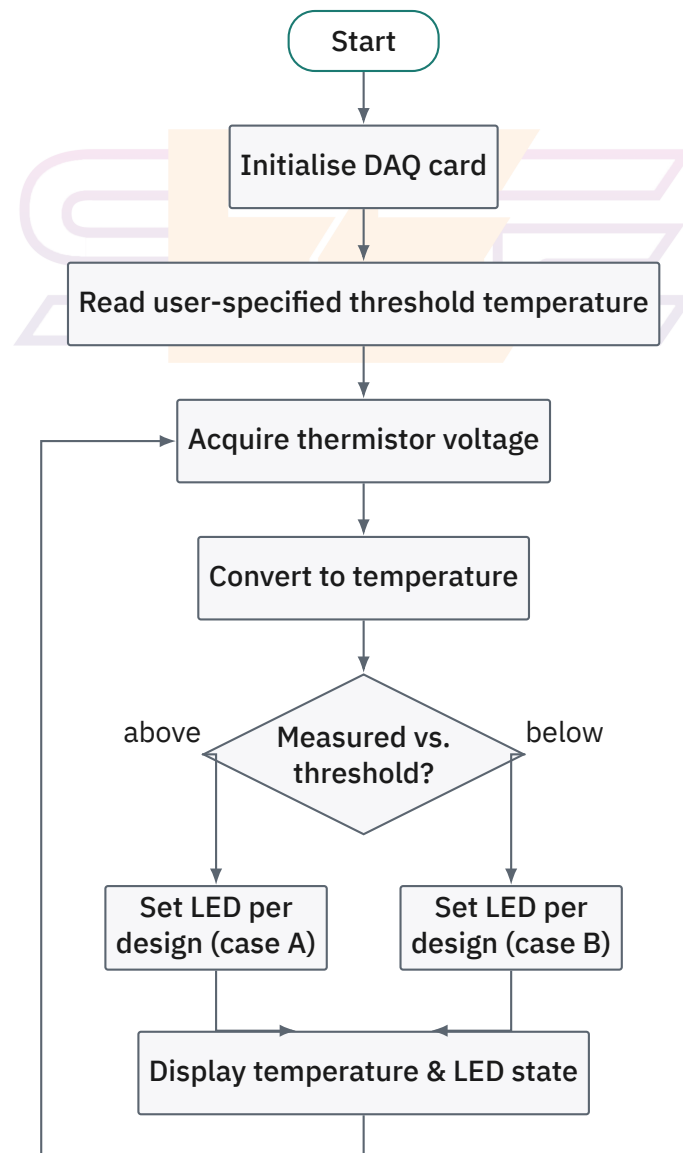


Figure 8.2: Temperature monitoring and LED control loop (ON/OFF direction per student design).

4. Set the controlled (threshold) temperature to a chosen value and record the measured temperature and LED state at 1-minute intervals for 35 minutes (Table 8.1). Plot temperature against time.
5. Repeat for at least two further threshold temperatures. Plot your results.
6. Discuss your findings, including the responsiveness of the LED control and the accuracy of your temperature conversion.

Table 8.1: Temperature and LED-state log.

Threshold temp.	Time (min)	Measured temp.	LED state
	1		
	2		
	...		
	35		

Report

1. Present your signal-conditioning circuit design (topology, calculations, component values) and its calibration against a reference temperature measurement.
2. State and justify your chosen LED ON/OFF logic (Activity 3, step 2).
3. Present and discuss your temperature-vs-time plots for each threshold tested.
4. What sources of error or lag affect the accuracy and responsiveness of this system (thermistor thermal mass, signal-conditioning bandwidth, software polling rate, etc.)?



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 9

PROJECT CREATION IN SCADA (IOT)

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 9

Project Creation in SCADA (IoT)

Objectives

1. To understand the role and capabilities of SCADA software in an industrial IoT system.
2. To create a working SCADA project, including a Project Node and a SCADA Node.
3. To configure a communication link (Comport) between the SCADA node and an automation device.
4. To create and configure IO Tags and Internal Tags for real-time data acquisition.
5. To configure basic alarms on analog and discrete tags.

Equipment / Software Required

- PC/laptop with Advantech WebAccess/SCADA installed (Project Node and SCADA Node)
- Siemens S7-1200 PLC (or equivalent), with an existing ladder-logic program and known I/O/DB addressing
- Ethernet cable and a configured local network connection between the PC and the PLC
- TIA Portal (or equivalent Siemens engineering software) for reference to the PLC's tag addresses

Introduction

Supervisory Control and Data Acquisition (SCADA) software sits between the plant floor and the people (or systems) that need to monitor and control it. It collects real-time data from automation devices — PLCs, controllers, direct digital control (DDC) systems, and distributed control systems (DCS) — and presents that data through live displays, trends, alarms, and reports.

Advantech WebAccess/SCADA, used in this experiment, is a 100% web-based SCADA package: projects are developed and accessed through a standard web browser, rather than a dedicated desktop client. This lowers the barrier to remote monitoring and multi-site deployment, and it integrates directly with IoT dashboards and cloud platforms via MQTT.

A WebAccess/SCADA system is built from three components:

- **Project Node** — the development platform and web server. All project configuration, graphics, and database files are stored here, and every client connects to it first.
- **SCADA Node** — runs the real-time Kernel (`datacore.exe`), which uses communication drivers to talk to automation hardware (PLCs, I/O modules, controllers) and supplies live data to clients.
- **ViewDAQ Client** — the browser-based interface used to monitor and control the SCADA node: viewing live graphics, historical trends, and alarms, and adjusting set points.

In a small setup (as in this lab), the Project Node and SCADA Node commonly run on the same machine.

Data inside WebAccess/SCADA is represented as **Tags**. There are two broad categories:

- **IO Tags** — real-time measurements and outputs read from, or written to, an automation device. IO Tags can be Analog (floating-point), Discrete (an integer 0–7, commonly used as a 0/1 digital value), or Text (an ASCII string up to 72 characters).
- **Internal Tags** — values that exist inside the SCADA software itself rather than being read from a device: **Constant** tags (a user-defined fixed or operator-editable value), **Calculation** tags (a mathematical/logical expression over up to 20 other tags), and **Accumulation** tags (an integration/totaliser over another tag, e.g. totalising flow into volume).

Both tag categories share the same alarm, security, logging, and reporting features once created.

Table 9.1: Example Siemens S7-1200 address mapping to WebAccess tag addressing.

Siemens S7-1200 address	Data type	Length (bits)	WebAccess address
%I0.4	Bool	1	IX000 (start bit 4)
%Q0.3	Bool	1	QX000 (start bit 3)
%M0.4	Bool	1	MX000 (start bit 4)
%MW0	UInt	16	MW000 (start bit 0)
%IW6.4	UInt	16	IW006 (start bit 4)
%DB1.DBX2.0	Bool	1	DBX1.2 (start bit 0)
%DB3.DBW10	UInt	16	DBW3,10
%DB3.DBD14	Real	64	DBD3,14

Note: for tags addressed inside a Data Block (DB), the offset must be enabled in the block's properties in TIA Portal, since WebAccess addresses DB tags by block number plus offset.

Procedure

Activity 1 — Create a new SCADA project

Objective: create a Project Node and SCADA Node in WebAccess/SCADA.

1. Open a web browser and navigate to 127.0.0.1 (or localhost) to reach the WebAccess/SCADA landing page as in Figure 9.1.

Default Account: admin

Default Password: (blank)

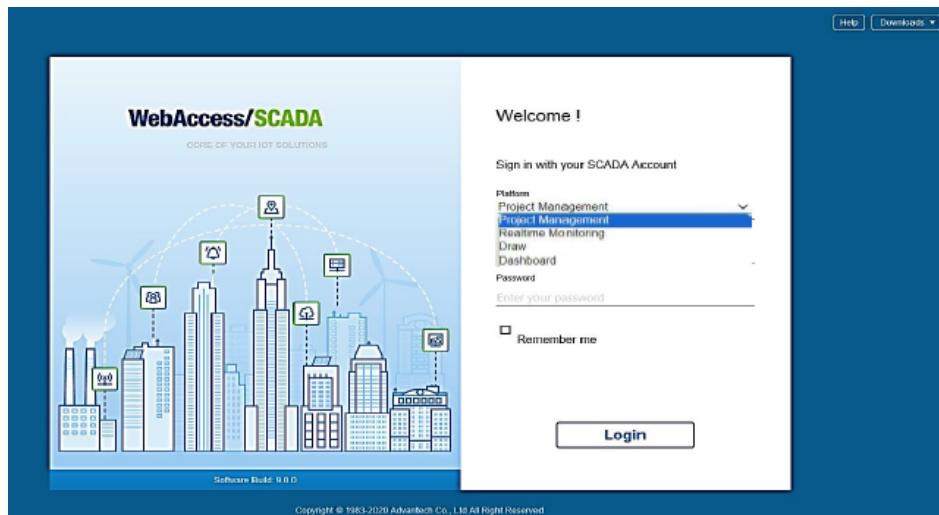


Figure 9.1: WebAccess landing page

2. Create a new project: enter a **Project Name** and **Project Description**, and set the **Project Node IP Address** (use 127.0.0.1 for a local machine, or the PC's hostname) as in Figure 9.2.

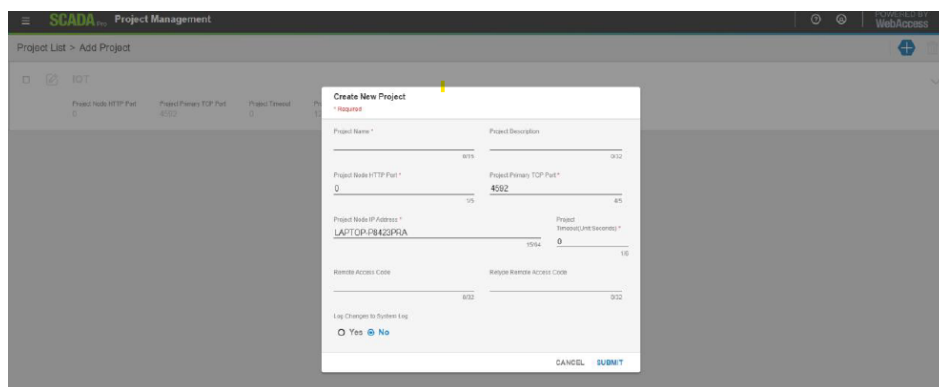


Figure 9.2: New project creation.

3. Leave the **Primary TCP Port** (default 4592) and **HTTP Port** (default 80) at their defaults unless your network/firewall requires otherwise, then click **Submit**.
4. Select the Project Node in the tree structure, click **+**, and choose **New Node** to create the SCADA Node. Fill in the Primary Node Name, Primary/Secondary TCP Port, Node Timeout, and SCADA Node IP Address. Refer to Figure 9.3.

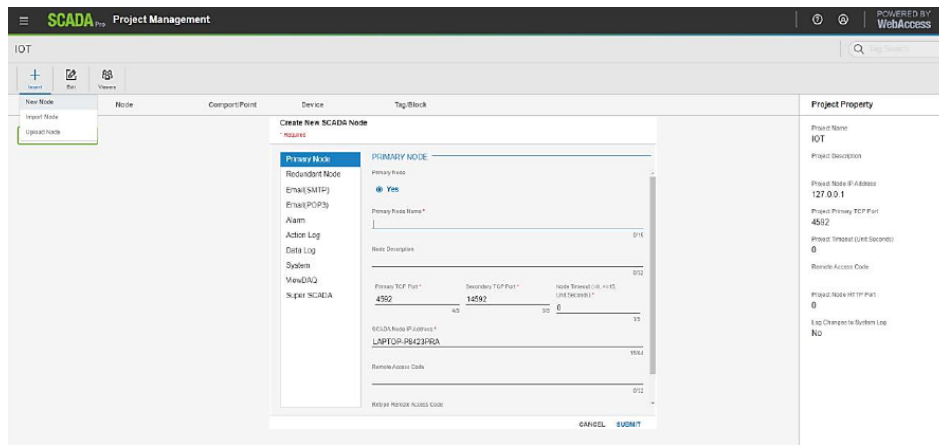


Figure 9.3: New project creation.

5. Start the Kernel on the SCADA Node (**Start Kernel** from the SCADA Node menu) and confirm it is running by hovering over the WebAccess/SCADA icon, which should report the running project and node name.

Activity 2 – Configure a communication port and add a device

Objective: create the physical/virtual link (Comport) between the SCADA node and the PLC, and add the PLC as a device on that port.

1. Under the SCADA Node, add a new **Comport**. Select the interface type appropriate to your connection:
 - **Serial** – an RS232/RS422/RS485 port (COM1, COM2, ...) on the SCADA node PC.
 - **TCP/IP** – the most common choice for PLCs and IoT devices; WebAccess/SCADA searches all TCP/IP network cards regardless of the assigned Comport number (Figure 9.4).

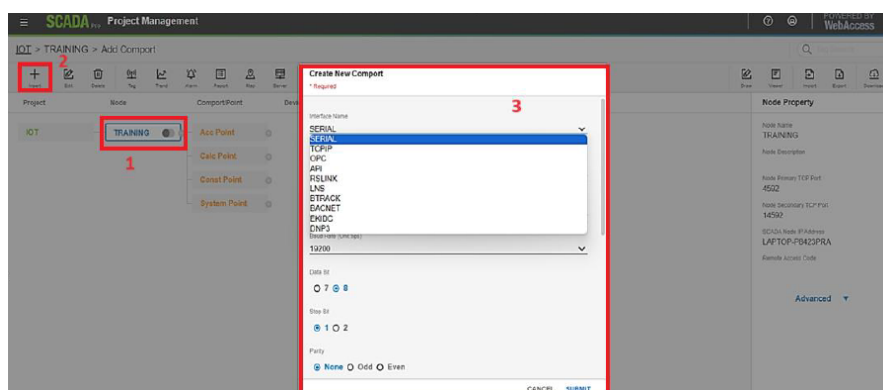


Figure 9.4: COMPort selection.

- **OPC** – via an OPC Server, for virtual/software-based links.
2. Configure the port: Port Number, Scan Time (e.g. 1000 ms), and Auto-recover Time.
 3. Under the configured Comport, add a new **Device**, selecting the correct Device Type/Driver for your hardware (e.g. Siemens S7-1200, Modbus TCP, Allen-Bradley) as in Figure 9.5.

- Enter the device's communication parameters (IP address, station ID, or baud rate for serial links) and save.

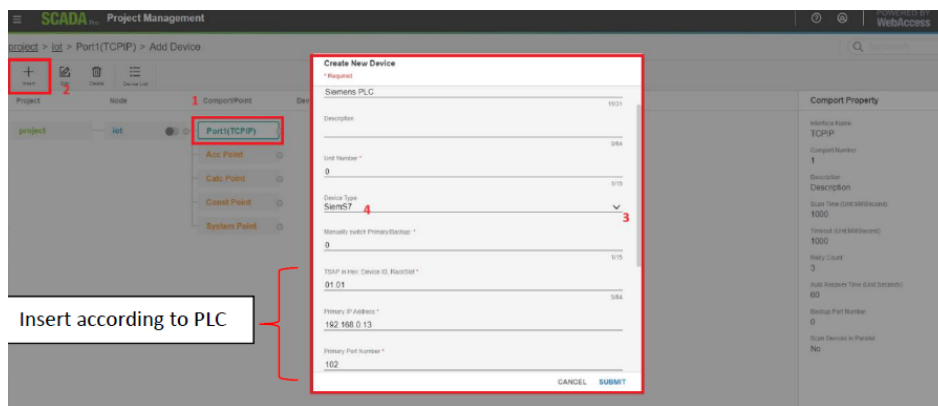


Figure 9.5: s7s-1200 device configuration.

Activity 3 – Create IO Tags

Objective: create IO Tags that map to real signals in the PLC program.

- Identify at least four signals in your PLC program to bring into SCADA — for example, a Start pushbutton, a Stop pushbutton, an Emergency-stop input, and one analog or word-type value (e.g. a counter or measured value).
- For each signal, determine its native PLC address and convert it to the equivalent WebAccess address using Table 9.1 as a reference (e.g. %DB1.DBX0.1 for a Start pushbutton, %DB1.DBX0.3 for Stop, %DB1.DBX0.0 for E-stop).
- Under the Device, click the tag icon and **Insert** a new tag for each signal. Fill in the Tag Type, Tag Name, Address, Start Bit, Conversion Code, and Length fields to match your PLC addressing.
- Save your tags, then verify each one updates correctly by forcing the corresponding input/output in the PLC and observing the tag's live value in WebAccess.

Activity 4 – Create Internal Tags

Objective: create at least one Constant Tag and one Calculation Tag.

- Under **Const Point** on the SCADA Node, insert a new Constant Tag (analog type, ConAna, or discrete type, ConDis) — for example, a user-adjustable setpoint value.
- Under **Calc Point** on the SCADA Node, insert a new Calculation Tag. Choose an analog or discrete result type, then write a mathematical/logical expression referencing one or more of your IO Tags (tags are referenced in the formula by their single-letter designations; the expression is limited to 80 characters).
- Save and verify that the Calculation Tag's value updates correctly as its input tag(s) change.

Activity 5 – Configure a basic alarm

Objective: enable alarming on at least one analog tag and one discrete tag.

1. Open the configuration for one of your Analog IO Tags and enable a non-zero alarm priority. Analog tags support High-High, High, Low, Low-Low, Deviation, and Rate-of-Change alarms — choose whichever is appropriate to the signal you selected.
2. Open the configuration for one of your Discrete IO Tags and enable state alarming (e.g. alarm when the E-stop input becomes active).
3. Trigger each alarm condition on the PLC side and confirm that the alarm appears correctly in WebAccess.

Report

1. List the Project Node and SCADA Node settings you used (IP address, ports, node names), and explain the role each plays in the system.
2. Present your Comport and Device configuration, including which interface type (Serial/TCP-IP/OPC) you selected and why it was appropriate for your hardware.
3. Tabulate the IO Tags you created: PLC address, WebAccess address, tag type, and a description of what each represents.
4. Present your Constant Tag and Calculation Tag configurations, including the expression used for the Calculation Tag and what it computes.
5. Describe the alarm(s) you configured and demonstrate (with a screenshot or description of your test) that each triggers correctly.
6. Discuss one advantage of WebAccess/SCADA's 100% web-based architecture compared to a traditional desktop-client SCADA system.

References

- MTA SkillLab Sdn Bhd, *i-Factory Training Manual — IIoT System Development Level 1*, Module 3: Project Creation in SCADA.
- Advantech WebAccess/SCADA product documentation: <https://www.advantech.com/en/support/details/installation?id=1-MS9MJV>



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 10

DASHBOARD VISUALIZATION AND ANALYSIS (IOT)

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 10

Dashboard Visualization and Analysis (IoT)

Objectives

1. To create a dashboard and populate it with multiple panel types.
2. To connect a dashboard panel to a database (ODBC) data source.
3. To configure an alert with a notification channel on a dashboard panel.
4. To bundle multiple dashboards into a shareable SRP-Frame.

Equipment / Software Required

- PC/laptop with access to the WebAccess/SCADA project created in Experiment 9
- WISE-PaaS/Dashboard (or the integrated Dashboard module within WebAccess/SCADA)
- A configured SCADA project with at least one live IO Tag and one Internal Tag (from Experiment 9)
- PostgreSQL (or another supported ODBC-compatible database), if historical/database-backed panels are required

Introduction

Once a SCADA project is collecting live data, that data still needs to be presented in a way people can act on quickly — an operator glancing at a screen, or a manager reviewing performance at the end of a shift, needs a different view than the raw tag list. This is the role of the **dashboard**: an HTML5-based, browser-accessible visualisation layer that turns tag data into graphs, single-value indicators, tables, and alerts.

In WebAccess/SCADA, dashboards are built from **panels**, each connected to one or more tags via a query. Commonly used panel types include:

- **Singlestat** — displays a single current value (e.g. current temperature, current speed), often with colour-coded thresholds.
- **Graph** — plots one or more tags over time, useful for trends.
- **Advance Datatable** — displays tabular data, useful for logging multiple related values together.
- **PictureIt** — overlays live tag values on top of an uploaded image (e.g. a process diagram or equipment photo), useful for giving values spatial context.

Dashboards can also connect directly to a database via **ODBC** (ready-made drivers exist for PostgreSQL, MySQL, SQL Server, and others), which allows a panel to query historical/logged data rather than only live tag values — useful for reports and trend analysis that go back further than the SCADA node's live buffer.

A dashboard on its own shows one view; when several dashboards need to be presented together as a single branded, navigable product — for an operator's shift-start screen, or a manager's overview — they can be bundled into an **SRP-Frame**: a wrapper that adds a title, logo, colour scheme, and a menu linking to each dashboard page, and can be shared as a single URL.

Procedure

Activity 1 — Create a dashboard and add panels

Objective: create a new dashboard and populate it with at least two different panel types, connected to live tags.

1. Log in to the Dashboard module (type 127.0.0.1 in the browser address bar, then select **Dashboard** from the WebAccess landing page; the default account is admin with no password).
2. From the + button on the left, create a new Dashboard page.
3. Add a **Singlestat** panel: select the panel type, choose the query and tag you want to display (use one of your IO Tags or the Calculation Tag from Experiment 9), then configure how the value is displayed (units, thresholds, colour).
4. Add a **Graph** panel: select the panel type, then select the query and tag(s) to plot over time.
5. Add at least one further panel of your choice (Advance Datatable or PictureIt), and configure it similarly.
6. Save the dashboard and confirm each panel updates as the underlying tag values change.

Activity 2 — Connect a panel to a database (ODBC)

Objective: add a database as a data source and build a panel that queries it.

1. From the Configuration menu, select **Data Sources**, then click **Add data source**.
2. Select the appropriate database type (e.g. PostgreSQL) and enter the connection details (host, port, database name, credentials), then click **Save & Test** and confirm the connection succeeds.
3. Create a new panel that queries this data source directly (rather than a live SCADA tag), and configure it to display the queried data.

Activity 3 – Configure an alert and notification channel

Objective: trigger a notification when a panel's value crosses a threshold.

1. On one of your panels, click the bell icon to open its Alert configuration.
2. Name the alert, and set the evaluation frequency and the **For** duration (how long the condition must persist before the alert fires).
3. Choose the evaluation expression appropriate to your signal (`Last()` for the latest value, `Avg()`/`Sum()`/`Max()`/`Min()`/`Count()` for aggregated values over the query window) and set the threshold condition.
4. Under the bell icon, open **Notification Channels** and click **Add channel**. Choose a notification type (e.g. Email), and configure whether it sends on all alerts, includes an image, and whether reminders are sent (and at what interval).
5. If using Email, configure the SMTP server settings under **Server Admin** → Settings.
6. Trigger your alert condition and confirm the notification is received.

Activity 4 – Build an SRP-Frame

Objective: bundle at least three dashboards into a single branded, navigable SRP-Frame.

1. Create at least two further dashboards (so you have a minimum of three in total), each on a different topic or view of your process.
2. From the SRP-Frame tab, click **Manage**, then **+SRP-Frame**, and enter a project name to create the frame.
3. In the **General** tab, set the title/description, upload a logo, and adjust the title, background, and menu colours.
4. In the **Menu List** tab, add a menu entry for each of your three (or more) dashboards, linking each entry to the correct dashboard page.
5. Save your changes, then preview the SRP-Frame via the **Main** button. Confirm you can navigate between all linked dashboards, and note the shareable URL.

Report

1. List the panels on your dashboard, their type, and which tag or data source each is connected to.
2. Present your ODBC data source configuration and describe what the database-connected panel displays.
3. Describe your alert configuration (evaluation expression, threshold, duration) and your notification channel setup, and confirm (with evidence) that the notification fired correctly.

4. Present your SRP-Frame: its menu structure, and a brief description of what each of your three (or more) dashboards shows.
5. Discuss one situation where an SRP-Frame (multiple linked dashboards) would be more useful to an operator than a single dashboard, and why.
6. Reflecting on Experiments 9 and 10 together, describe the full data path from a physical PLC input to a value displayed on your dashboard.

References

- MTA SkillLab Sdn Bhd, *i-Factory Training Manual – IIoT System Development Level 1*, Module 4: Dashboard Viewing and Analysis.
- Advantech WISE-PaaS/Dashboard product documentation.

